UNIVERSITY
OF OSLO

**Master's thesis**

# Towards Automated Language Server Synthesis

Building Language Servers by Extracting Grammars and Semantics from Code Examples

**Jonas Lien Sampaio da Silva**

Programmering og systemarkitektur
60 ECTS study points

Department of Informatics
Faculty of Mathematics and Natural Sciences




Spring 2025

**Jonas Lien Sampaio da Silva**

# Towards Automated Language Server Synthesis

Building Language Servers by Extracting Grammars and Semantics from Code Examples

Supervisors:
Volker Stolz, Shaukat Ali and Erblin Isaku

**Abstract**

Modern software developers expect rich code intelligence, such as completion, go-to-definition, and reference search, regardless of the language. Yet, creating Language Server Protocol (LSP) servers that provide these features demands significant expertise, leaving many domain-specific and niche languages underserved. This thesis introduces **uLangs**, a toolkit that automates the synthesis of core code intelligence components from code examples and leverages them to provide intelligent IDE features. uLangs achieves this by first defining a *Language Description Specification (LDS)*, composed of *ANTLR* grammars for parsing and *Prolog* adapters for semantic interpretation, then employing Large Language Models (LLMs) within an iterative *generate-verify-repair* process to automatically synthesize the LDS artifacts.

A case study on Lua demonstrated uLangs' practical feasibility. It achieved **80% accuracy** for "Go to Definition", outperforming a naive string-matching baseline. For "Find References", uLangs exhibited high recall (97.6%), indicating effective candidate identification. uLangs demonstrates a significant reduction in the effort required to bootstrap LSP support, offering a novel approach combining generative AI with formal language tooling to make advanced developer tooling more accessible.

# Contents

List of Acronyms . . . . . vii
1 Introduction . . . . . 1
1.1 Background and motivation . . . . . 1
1.2 Problem Statement . . . . . 1
1.3 Scope and Objectives . . . . . 2
1.4 Main contributions . . . . . 4
1.5 Methodology . . . . . 4

**I Background** **7**
2 Code Intelligence . . . . . 9
2.1 Language Server Protocol (LSP) . . . . . 9
2.2 Existing Language Workbenches . . . . . 10
3 Large Language Models (LLMs) . . . . . 13
3.1 Prompting and Reasoning . . . . . 14
3.2 Retrieval-augmented Generation . . . . . 14
3.3 Agents . . . . . 15
3.4 Reasoning models . . . . . 15
3.5 Structured Outputs . . . . . 16
4 ANother Tool for Language Recognition (ANTLR) . . . . . 17
4.1 Parser generation . . . . . 17
4.2 Actions . . . . . 17
5 Prolog . . . . . 19
5.1 Facts, Rules, and Queries . . . . . 19
5.2 Unification and Search . . . . . 20
5.3 Data Representation and Manipulation . . . . . 20
5.4 Relevance for Semantic Code Analysis . . . . . 20
6 Related work . . . . . 21
6.1 Grammars and Their Inference . . . . . 21
6.2 Semantic Understanding of Code . . . . . 24
6.3 Gaps in Automated Code Intelligence . . . . . 25

**II Implementation** **27**
7 System Overview . . . . . 29
7.1 Unified toolkit . . . . . 30
7.2 Designing a machine-readable language description schema . . . . 32

8 Syntactic analysis and synthesis . . . 35
8.1 Synthesizing ANTLR grammars . . . 35
8.2 Feedback signal . . . 36
9 Semantic analysis and synthesis . . . 39
9.1 Tree linearization . . . 39
9.2 Main queries . . . 40
9.3 Synthesizing an adapter . . . 42
9.4 Feedback signal . . . 43
9.5 Inter-file dependencies . . . 44
10 Universal LSP Server . . . 47
11 Key Algorithms . . . 49
11.1 Automated generation with Verifier . . . 49
11.2 Feedback-based repair loop . . . 50
11.3 Context Compression . . . 53
11.4 File selection, splitting and diversification . . . 54
12 Challenges and Trade-offs . . . 57
12.1 Choosing a parser generator . . . 57
12.2 Choosing SWI-Prolog . . . 59
12.3 Mitigating issues with example-driven artifact synthesis . . . 60
12.4 Advanced search algorithms . . . 61
12.5 Model choice . . . 63

**III Evaluation and Conclusion** **67**
13 Evaluation . . . 69
13.1 Metrics . . . 69
13.2 Evaluation Procedure . . . 71
13.3 Qualitative and Results . . . 71
13.4 Analysis and Interpretation . . . 77
14 Limitations and Future Work . . . 83
14.1 Extending Generality and Expressiveness . . . 83
14.2 Fine-tuning . . . 84
14.3 Enabling Production-Ready Tooling . . . 86
15 Conclusion . . . 89

**Appendices** **95**
A Prompts . . . 97
B JSON Schema . . . 101
C Case Study . . . 103
D Declaration of the use of generative artificial intelligence . . . 105

# List of Figures

1.1 High-level overview . . . . . 3

7.1 Inference mode pipeline . . . . . 30
7.2 Snippet-driven synthesis . . . . . 31

11.1 Automated generation with verifier flowchart . . . . . 52

# List of Tables


2.1 LSP Synchronization Methods . . . . . 10
2.2 Common LSP Methods. . . . . 10

9.1 Feedback in the semantic synthesis verification system . . . . . 43

13.1 Synthesis process statistics . . . . . 72
13.2 uLangs LSP Client Evaluation . . . . . 76
13.3 Naive LSP Client Evaluation . . . . . 76

# List of Acronyms

**ANTLR** ANother Tool for Language Recognition.

**AST** Abstract Syntax Tree.

**BNF** Backus-Naur Form.

**CFG** Context-Free Grammar.

**CST** Concrete Syntax Tree.

**IDE** Integrated Development Environment.

**LDS** Language Description Specification.

**LLM** Large Language Model.

**LSP** Language Server Protocol.

**MCTS** Monte Carlo Tree Search.

**RL** Reinforcement Learning.

# Preface

Completing this Master's thesis marks the end of an exciting and rewarding journey. My longstanding fascination with Artificial Intelligence and passion for improving developer experience initially led me to explore how Large Language Models could leverage Language Server Protocol methods. Interestingly, the project evolved into the work presented here through insightful discussions with my supervisor, Volker Stolz, and my colleague, Kristian Tjelta Johansen.

I am deeply grateful to my primary supervisor, Professor Volker Stolz, for his invaluable guidance and encouragement, and to my co-supervisors, Shaukat Ali and Erblin Isaku, for their support and valuable insights. Special thanks go to Kristian Tjelta Johansen for helping me crystallize the core ideas that inspired this thesis.

Finally, heartfelt thanks to my family and friends, especially those who shared the Master's thesis journey with me, for their constant support. Most importantly, thank you to my girlfriend for her patience, understanding, and all the late dinners endured while I worked towards completing this thesis.

This work would not have been possible without these individuals' collective encouragement and support.

# Chapter 1

# Introduction

## 1.1 Background and motivation

The field of software development continually sees the emergence and evolution of programming languages and domain-specific languages (DSLs). As developers encounter both new and established languages in their work, they naturally expect consistent and robust tooling support from Integrated Development Environments (IDEs), regardless of the language used. As developers experience the benefits of rich tooling—such as intelligent code completion, precise go-to-definition, and reliable automated refactoring—in one context, the expectation for similar support extends to other languages they employ. The very existence and adoption of mechanisms like the Language Server Protocol (LSP) (as discussed in Section 2.1) underscore this widespread demand for consistent and powerful "language intelligence" across different editors and languages.

However, providing such sophisticated tooling, especially through developing Language Servers, is non-trivial. Creating parsers, semantic analyzers, and the necessary integrations traditionally requires considerable expertise in language design and compiler construction principles, coupled with significant implementation effort. While this investment is often made for widely used, general-purpose languages, the associated costs and complexities can present a considerable barrier for newer, niche, or rapidly evolving languages. This disparity can lead to a "tooling gap," where developers working with these less mainstream languages may not have access to the same level of advanced IDE support, potentially impacting productivity and adoption.

This thesis is motivated by the desire to make sophisticated IDE support more universally accessible and to explore avenues for bridging this potential tooling gap. An initial investigation into the existing landscape of language engineering tools revealed a relative scarcity of solutions capable of fully automatically synthesizing the core syntactic and semantic components of a Language Server directly from a collection of code examples. This apparent gap, which will be further explored in the context of related work (Chapter 6), underscored an interesting and pertinent research avenue: the possibility of developing a novel, automatic, example-driven approach to drastically lower the effort required to bootstrap rich code intelligence for any given language.

## 1.2 Problem Statement

Developers increasingly expect robust development tooling even as more programming and domain-specific languages emerge. Creating such tooling, particularly language

servers, involves several distinct and technically demanding stages, including syntax parsing, semantic analysis, and Language Server Protocol (LSP) integration. While components can sometimes be reused from existing compilers or interpreters, this is often not straightforward, as language designers and tooling developers may be distinct teams, and compiler internals are typically not designed with direct IDE integration in mind. Consequently, when existing tooling cannot be repurposed, developers must implement these components themselves. Successfully doing so requires a thorough understanding of the target language's grammar and semantics, coupled with knowledge of language tooling architecture and design principles. For niche languages or those with smaller communities, the need to address these multifaceted requirements can make providing comprehensive development environments a complex undertaking.

Existing tools address parts of this problem but leave gaps. Xtext [57] allows developers to define languages using high-level specifications and generates parsers, type systems, and IDE features. However, the language must be designed within its framework, making it difficult to adapt existing languages without significant modification. While it automates syntax and tooling generation, developers often need to refine semantic behavior manually, especially for complex languages. Tree-sitter [1] simplifies syntax parsing but lacks deep semantic integration, making it insufficient for complete language tooling. LLM models show promise in code generation but lack the precision and reliability needed for building complete language servers.

This thesis addresses these limitations by introducing a universal language server toolkit, called **uLangs**[1], a unified and automated approach to language server development. uLangs automatically synthesizes the necessary language server components to give the user of the toolkit a complete end-to-end language server. It combines grammar-based parsing (ANTLR) and semantic analysis with Prolog to analyze a target language and provide the essential code intelligence. By automatically synthesizing these intermediate artifacts (grammars and semantic adapters) from code examples, uLangs equips its runtime toolkit to deliver code intelligence, significantly lowering the barrier for providing advanced language support.

## 1.3 Scope and Objectives

The scope of this thesis includes the design, implementation, and evaluation of the uLangs toolkit, a unified system for automating the creation of language servers from code examples. To achieve this, we first define a structured, machine-readable Language Description Specification (LDS) consisting of ANTLR grammars for parsing and language-specific Prolog code for semantics-aware transformation of the parsed tree (we later call this the *Semantic adapter*). We then leverage Large Language Models (LLMs) to automatically synthesize these artifacts directly from provided code snippets. The synthesized grammars and adapters encapsulated by the LDS serve as the foundation for implementing standard Language Server Protocol features, such as symbol resolution, reference finding, and document structure queries. Given the exploratory nature of this work, we limit our focus to a representative subset of code intelligence features, leaving broader LSP functionality for future investigation. Finally, we evaluate the effectiveness of our approach through a case study, measuring both the accuracy of the synthesized artifacts and the reduction in development effort compared to traditional manual methods.

[1] All code is available at DOI: 10.5281/zenodo.15380780; or at github.com/jonaslsaa/ulangs.

Note that within the defined scope of this thesis, code intelligence capabilities are primarily demonstrated through *intra-file analysis*. Nevertheless, the underlying architecture of uLangs, particularly the concept of a modular Language Description Specification, is designed to facilitate future extensions towards project-wide, multi-file code intelligence, potentially through the introduction of a *Project Adapter* (as discussed in Section 9.5). The main objectives can be summarized as follows:

- **Design a Programming Language Specification**: Create a machine-readable specification capable of representing a diverse set of programming languages and designed to facilitate reliable LLM generation through explicit structural constraints.
- **Develop the uLangs Toolkit**: Design and implement a unified toolkit that can ingest code examples in any language and automatically produce a working parser and a semantics-aware transformation function, forming the core of an LSP server.
- **Automate Language Server Creation**: Enable end-to-end generation of a language server by integrating the synthesized parser and semantic logic into an LSP server framework, reducing the need for manual coding of editor features.
- **Qualitatively Evaluate Synthesis Feasibility and Quality**: Assess the practicality and effectiveness of the automated synthesis process by applying uLangs to a known programming language or DSL, qualitatively analyzing the correctness, completeness, and utility of the generated grammar and semantic adapter.

By defining the scope and these objectives, we establish clear boundaries for the research and concrete goals against which success can be measured.

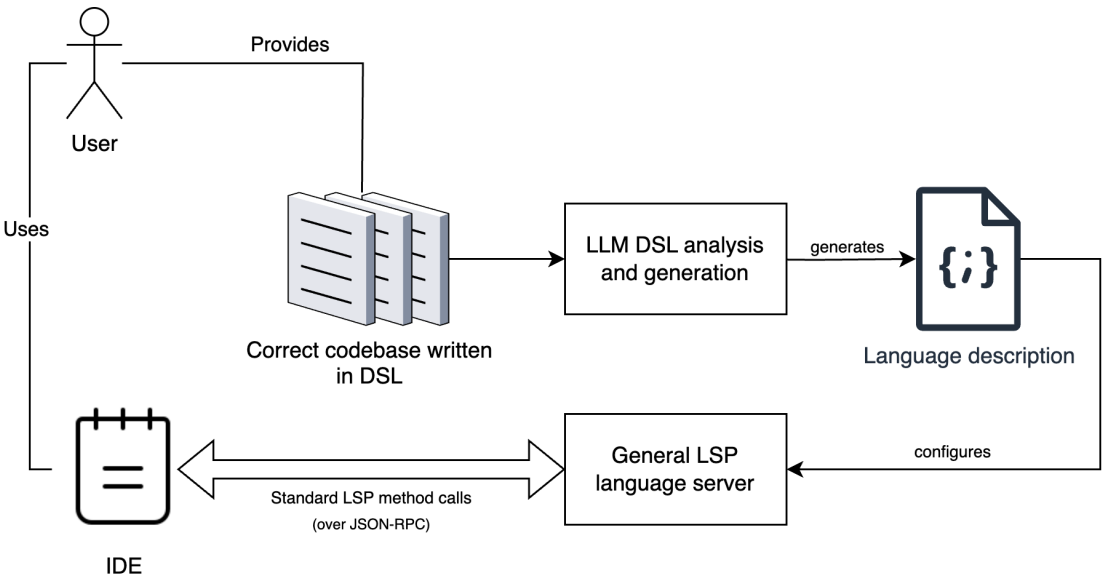



Figure 1.1: High-level workflow using the toolkit. An LLM analyzes user-provided DSL code to generate a language description, configuring a language server for IDE integration.

## 1.4 Main contributions

This thesis introduces uLangs, a novel end-to-end toolkit and methodology for automating the synthesis of language server components and code intelligence tools directly from code examples, marking significant contributions to language engineering and automated software development tooling through several interconnected, novel components:

1. A defined **Language Description Specification (LDS)**, which encodes a target language's syntactic and semantic rules. This specification is designed to be automatically and reliably generated by Large Language Models (LLMs) and used by the uLangs toolkit to provide specific code intelligence features.
2. A sophisticated **synthesis logic** that uses LLM within an iterative *generate-verify-repair* loop to automatically create an LDS for any programming language given.
3. **Inference logic** that utilizes the generated LDS to provide on-demand code intelligence, such as symbol resolution and reference finding, forming the core of an LSP server.

The feasibility and effectiveness of this approach were empirically validated through a detailed case study. This evaluation assessed the performance of the uLangs-synthesized code intelligence for Lua on two core LSP features: `textDocument/definition` (achieving **80% accuracy**) and `textDocument/references` (exhibiting high recall at 97.6% micro-averaged and an F1-score of 82.8%). **These results demonstrate that uLangs can automatically generate functional and useful language tooling components from examples**, highlighting its current capabilities and areas for future refinement in semantic precision.

Ultimately, this research demonstrates that the novel combination of LLM-driven synthesis, a structured and synthesizable LDS, and an iterative refinement process represents a significant step towards more fully automating the creation of sophisticated language intelligence tools. uLangs showcases a promising pathway to drastically reduce the manual effort and specialized expertise traditionally required by successfully synthesizing both parsing and semantic analysis capabilities from code examples alone. This, in turn, makes rich development support more accessible for a broader spectrum of programming languages and domain-specific languages, contributing to the broader goal of enhancing developer productivity and language adoption.

## 1.5 Methodology

The development and evaluation of the uLangs toolkit involved iterative design, implementation of a novel software artifact, and empirical assessment through a targeted case study. The overall methodology consisted of several key phases: first, a literature review and problem analysis to define the tooling gap and existing limitations clearly; second, the conceptual design and architecture of the uLangs system, integrating grammar synthesis, semantic adapter generation, and a universal LSP server; third, iterative implementation and prototyping, leveraging LLMs for automated artifact synthesis, combined with formal language tools such as ANTLR and Prolog; and finally, evaluation through a case study to assess the effectiveness, generality, and limitations of the synthesized language tooling. Specific details of the research method, experimental setup, and evaluation approach are elaborated later.

### 1.5.1 Research Method and Rationale

Although the initial development of the *uLangs toolkit* was driven by practical engineering goals, the research process aligns closely with the *Constructive Research Approach* [25]. Constructive research aims to solve practical problems through the design and development of novel solutions (such as models, methods, or software systems). The research contribution lies both in the solution itself and in the insights gained during its construction and evaluation.

The constructive research approach was chosen because it aligns closely with the central goal of this thesis: the creation of uLangs, a novel software toolkit designed explicitly to address the practical challenge of automating language server development. The problem of the "tooling gap" identified earlier requires a concrete solution in the form of a new system, making a solution-centric approach particularly suitable. Furthermore, constructive research supports the types of contributions targeted by this thesis, including the uLangs toolkit itself, the novel synthesis algorithms developed, and the empirical evaluation of its effectiveness in generating robust language tooling.

In line with this approach, the research involved iterative cycles of problem analysis, solution design (including the uLangs architecture and synthesis algorithms), implementation, and evaluation via a case study on a real-world programming language.

# Part I

# Background

# Chapter 2

# Code Intelligence

## 2.1 Language Server Protocol (LSP)

In 2015, Microsoft introduced the Language Server Protocol (LSP) for their Integrated Development Environment (IDE), Visual Studio Code. Later in 2016, Microsoft started to work on making the LSP an open and generic protocol, allowing it to be used between any IDEs and Language Servers. The purpose of the protocol is to provide "language intelligence". Before LSP, language developers or IDE developers would have to support specific IDEs through their extension API explicitly. Thus increasing friction in extension development for new languages, as well as in the creation of new IDEs. Additionally, language developers were forced to use the programming language dictated by the IDE, rather than having the flexibility to choose the language of their preference for implementing language support. Consequently, the quality of language support for specific languages varied greatly across different IDEs. [6]

The LSP works as a middleware between a Language Server (LS) and an IDE, or another software needing code intelligence, and enables capabilities like auto-complete, go-to definition, and diagnostics (compiler error or warning). This "middleware" means that the Language Server is decoupled from the IDE, leading to several advantages [6]:

- As mentioned, it can be developed in any language separate from the IDE.
- Language Server runtime is separate from the IDE runtime. Therefore, crashes and slowdowns don't affect the IDE, and the IDE can easily restart or hot-swap the LS when needed.
- The Language Server can be reused across multiple IDEs and platforms, reducing the development effort and increasing the consistency of language support.

### 2.1.1 LSP Communication Mechanism

The communication of LSP is done by exchanging JSON-RPC messages, where the capabilities of the LS are modeled as Remote Procedure Call (RPC) *methods*. As the LSP specification does not dictate specific transport channels for message transmission, it is up to the client and server to decide. Common transport channels include sockets, standard I/O, pipes, and inter-process communication (IPC). This flexibility allows LSP components to be deployed in various configurations, such as within the same process, as a standalone process, or even on a separate machine. In practice, however, it is most commonly implemented as a standalone process on the same machine [37].

The Language Server Protocol (LSP) establishes a way for clients and servers to initialize communication, which allows them to negotiate and exchange capabilities, enabling the protocol to adapt to varying levels of support. This means that even if a client or server doesn't support all protocol features, LSP's flexible framework ensures seamless communication and feature discovery.

As discussed, one of the benefits of the LSP standard is a separation of concerns between the code editor and language server. The LS is responsible for parsing, analyzing, and diagnosing errors [6] in the code provided by the editor. For this to be possible, the LSP specification defines several synchronization methods to keep both programs in sync and avoid a complete re-process of the files on each request in the LS. Some of these methods are as follows:

| Method | Explanation |
|---|---|
| `textDocument/didOpen` | Notifies the LS that a new file has been opened in the editor. |
| `textDocument/didChange` | Notifies the LS that the contents of a file have changed. |
| `textDocument/didClose` | Notifies the LS that a file has been closed in the editor. The LS can then release any resources associated with the file. |

Table 2.1: LSP Synchronization Methods

After initialization and synchronization of open files, the client (code editor or IDE) can start querying the server for *code intelligence*. The LSP specification defines several methods for this purpose, some of which are as follows:

| Method | Explanation |
|---|---|
| `textDocument/hover` | Provides information about the symbol at the cursor position, such as its definition, type, and documentation. |
| `textDocument/completion` | Offers a list of possible completions for the current code snippet, including function and variable names, keywords, and more. |
| `textDocument/documentSymbol` | Returns a list of symbols in the current document, including functions, variables, classes, and interfaces, along with their locations and scopes. |
| `textDocument/definition` | Finds all references to a symbol in the current document, allowing the user to see where a function, variable, or class is used. |

Table 2.2: Common LSP Methods

## 2.2 Existing Language Workbenches

While creating language-specific tooling often involves significant manual effort, several established language workbenches offer more integrated solutions for this task. Prominent examples like Eclipse Xtext, Spoofax, and JetBrains MPS provide frameworks designed to automate aspects of language creation and IDE support generation. Each of these tools provides distinct approaches and trade-offs in language engineering,

particularly regarding how languages are defined, how tooling is generated, and how developers interact with the resulting languages.

Eclipse Xtext is built around a grammar-driven approach. At its core is a specialized grammar language, designed explicitly for describing textual languages. Developers define the concrete syntax of their language and specify how this syntax maps onto an underlying semantic model. From this grammar specification, Xtext automatically generates parsers, abstract syntax trees (ASTs), and various IDE features such as syntax highlighting, validation, and code completion. Additionally, Xtext supports grammar reuse through a mechanism called grammar mixins [56], allowing developers to extend existing grammars. While powerful, this approach requires developers to explicitly define and maintain formal grammar specifications, which can be challenging when adapting existing languages or rapidly prototyping new ones.

Spoofax also emphasizes declarative language definitions, but it does so with a broader set of specialized meta-languages, each tailored to a specific aspect of language design. For syntax definition, Spoofax uses the Syntax Definition Formalism (*SDF3*[45]), which supports generalized parsing, modular grammar definitions, and built-in mechanisms for disambiguation and error recovery. Beyond syntax, Spoofax offers separate meta-languages for defining other important language features. For example, developers can specify rules for name resolution and type checking using *Statix*[46], describe how data flows through programs with *FlowSpec*[44], define code transformations such as optimizations or compilation steps using *Stratego*[47], and configure editor features like syntax highlighting, reference resolution, and hover tooltips using the Editor Service Specification language (*ESV*[43]). From these high-level definitions, Spoofax automatically generates comprehensive tooling, including parsers, type checkers, interpreters, and IDE plugins for Eclipse and IntelliJ [42]. This declarative approach allows language designers to concentrate on the essential aspects of their language rather than specific implementation details. However, the need to learn and manage multiple specialized meta-languages can make Spoofax challenging to adopt, especially for developers new to its ecosystem.

JetBrains MPS takes a fundamentally different approach known as *projectional editing*. Instead of parsing textual source code, MPS directly manipulates the abstract syntax tree (AST), eliminating parsing ambiguities completely [19]. This approach allows developers to seamlessly embed and combine multiple languages within a single environment. However, since projectional editing introduces a distinct interaction model that diverges significantly from traditional text-based editing workflows, it makes the barrier to entry very high for all users of the DSL in question. Projectional editing inherently bypasses the parsing of textual source code, which is a critical component of the Language Server Protocol (LSP) workflow targeted by this thesis.

These existing language workbenches significantly reduce the manual effort involved in creating language tooling, but they still require considerable upfront manual work and knowledge within their specific ecosystems and constraints. Developers must explicitly define formal specifications or adopt particular editing paradigms, limiting flexibility and making it challenging to quickly adapt existing languages or integrate tooling into diverse environments. This highlights a clear opportunity for approaches that can even further automate language tooling creation in a more ecosystem-agnostic manner, particularly for existing DSLs, optimally circumventing the need to fully adopt and master a specific framework.

# Chapter 3

# Large Language Models (LLMs)

In recent years, the machine learning field of natural language processing (NLP) has exploded with the advent of Large Language Models (LLMs), like ChatGPT by OpenAI. LLMs are deep neural-network-based models trained on vast amounts of text data, enabling them to learn complex linguistic patterns. LLMs have demonstrated remarkable capabilities in replicating coherent human-written text, and with either clever prompting or fine-tuning can excel in various NLP tasks, such as language translation, text generation, and question-answering [18]. The most common LLM model architecture is the Generative Pre-training Transformer (GPT), which leverages attention mechanisms to capture long-range dependencies in language. This marked a significant breakthrough in the field of language models compared to previous approaches [36].

As shown by Brown et al. [8] (OpenAI), pre-trained GPT models are capable of learning from a tiny amount of data (few-shot learning) and sometimes even without any demonstrative data at all (zero-shot learning). [1] This means that they can perform well on tasks without any fine-tuning or training on the specific task. This is possible because of the vast amount of data they are trained on, which enables them to learn general language patterns. [8] However, task-specific fine-tuning can yield substantial performance gains, enabling even smaller models to surpass their larger counterparts in certain scenarios [41].

The landscape of Large Language Models (LLMs) has also undergone rapid expansion, with the emergence of numerous novel models, some proprietary and others open-source (OSS), most of which are grounded in the GPT architecture. These models exhibit varying levels of performance and quality. Some notable examples include:

- GPT-4 series and o-series models by OpenAI (closed)
- Claude models by Anthropic (closed)
- Gemini models by Google (closed)
- Llama models by Meta (open weights)
- Mistral models (open weights)
- DeepSeek models (open weights)

[1] Few-shot learning (or in-context learning) means performing well on a new task with only a few examples. In contrast, zero-shot learning means performing tasks without examples, relying solely on pre-trained knowledge and given instructions. This ability to adapt to new tasks and domains with minimal training is particularly impressive in LLMs.

These models, including their community-driven fine-tunes, have collectively advanced the state-of-the-art in LLM technology, with their applications continually expanding into diverse domains, including but not limited to code generation, language translation, and sentiment analysis.

The primary ways to utilize these models are through text completion and conversational interactions (also known as *chat* or *instruct*). Training a model on large text datasets yields a base model, which can be used for text completion tasks. To enable conversational capabilities, the base model can be fine-tuned on datasets of chat exchanges[2], these are delimited by some markers (called special tokens). This fine-tuning process allows the model to respond directly to user queries in a conversational style and follow instructions accurately [59]. The resulting "chat" or "instruct" models are often preferred for out-of-the-box use, with "instruct" models typically designed for single-turn conversations that focus on following user instructions in a more formal tone, whereas "chat" models may exhibit more personality, although these terms are sometimes used interchangeably.

A few different metrics and characteristics can be used to classify models. These metrics can be categorized by performance and quality. We want models to ingest and generate tokens[3] fast, have large context windows, and have a small memory footprint (GPT models are notorious for how much memory they require). Arguably, more importantly, we want models to follow our instructions carefully and generate answers of high quality, which are measured by a large array of evaluation benchmarks like *MMLU*, *MT-Bench*, and *LLMEval* to name a few [11].

## 3.1 Prompting and Reasoning

Prompting is a technique used to elicit specific behaviors or responses from Large Language Models (LLMs) by providing carefully crafted input text. This method leverages the pre-trained knowledge of the model to perform various tasks without additional fine-tuning. One advanced form of prompting is Chain-of-Thought (CoT) reasoning, which enhances the model's ability to handle complex problems by guiding it through a step-by-step reasoning process.

Chain-of-Thought (CoT) reasoning involves breaking down a problem into a series of intermediate steps, allowing the model to tackle each part sequentially before arriving at a final solution [55]. This approach is particularly useful for tasks that require multi-step reasoning, such as mathematical problem-solving, logical inference, and complex question answering. Wei et al. [55] demonstrated that CoT prompting significantly improves the performance of LLMs on tasks requiring complex reasoning. By explicitly instructing the model to think through each step, CoT prompting helps mitigate common issues such as hallucinations and erroneous conclusions.

## 3.2 Retrieval-augmented Generation

Although powerful, these models suffer from a few limitations, one of the most prominent being *hallucinations* - where the model generates plausible-sounding but outdated or factually incorrect text. Recent advancements have focused on Retrieval-augmented Generation to address hallucinations, where the language model is combined with an

[2] Additionally, a subsequent training step often involves applying Reinforcement Learning from Human Feedback (RLHF) to incentivize the model to exhibit "helpful and harmless" behavior [2, 59].

[3] In the context of LLMs, a *token* refers to a single unit of text, such as a word, character, or subword.

information retrieval system, where relevant text is inserted into the model's context to ground the generation in factual documents. This has been shown to increase the model's accuracy and enable new capabilities [16].

## 3.3 Agents

We have seen that state-of-the-art (SOTA) models exhibit remarkable abilities in a large number of tasks across many domains. [10] Although impressive, this ability deteriorates as tasks grow in complexity and start to require multi-step problem-solving.

Agents refer to AI-driven autonomous entities that are implemented on top of the model to perform specific tasks or achieve certain larger goals, but have been a concept in the field for many years [39]. Cheng et al. [12] compile a set of characteristics agents generally have:

- **Autonomy:** Agents independently perceive their environment, make decisions, and take actions without relying on external instructions.
- **Perception:** Agents are equipped with sensory capabilities that allow them to gather information about their environment through the use of sensors.
- **Decision-making:** Agents make decisions based on perceived information, selecting appropriate actions to achieve their goals.
- **Action:** Agents perform actions that alter the state of their environment. This is also commonly called *Tool Use* in practice.

In summary, agents should work without human intervention with a specific user-defined goal in mind, where they plan their actions, gather new information, make decisions, and finally take action. These steps should work as a feedback loop until the agent decides it has achieved its goal.

More advanced agentic systems can be implemented with several agents, either in *collaborative* or *hierarchical* arrangements, and have been shown to be effective in solving complex tasks [50].

## 3.4 Reasoning models

In late 2024 and early 2025, a new category of LLMs was introduced, often referred to as reasoning or thinking models. These models spend time thinking about the user query before finally answering. The first official release was OpenAI's o1 model release [33], but many subsequent releases have improved upon this model. This is achieved by generating an arbitrary amount of "thinking" tokens that aren't a part of the final answer, the model is usually trained via reinforcement learning (RL) on verifiable tasks learning to use more tokens to explore the problem space and find better solutions, which significantly improve domains like math and code. Conceptually, this can be viewed as an extension of the Chain-of-Thought (CoT) prompting strategy discussed earlier. By enabling the model to consider multiple reasoning paths and edge cases, these thinking models excel, especially at generating complex logic and code.

In 2025, newer reasoning models from various AI labs have been released, which are both more affordable, faster, and more token-efficient. Deepseek's R1 model showed that o1's performance trained [15] and served at a fraction of the cost [14, 31]. OpenAI also released their second-generation reasoning model, called *o3-mini* [29], which outperforms

prior models while being cheaper and more token efficient [3], meaning they use fewer tokens while "thinking" while still achieving high performance in benchmarks.

## 3.5 Structured Outputs

Structured Outputs [26] in Large Language Models (LLMs) is a feature that enables the model to generate JSON outputs that strictly conform to a specified JSON schema. Unlike earlier approaches where LLMs might produce inconsistent or invalid JSON when prompted, Structured Output restricts the tokens the model can generate, ensuring the output always adheres to the schema. This capability is vital for applications requiring reliable, machine-readable data, such as data extraction or API integrations. Structured Outputs can be used by specifying a JSON schema when using the OpenAI API.

The OpenAI API is widely regarded as an industry standard and has influenced other providers to adopt the same functionalities. However, although many providers choose to use an OpenAI-compatible API, not all features, including Structured Output, are consistently available across different providers, reflecting variations in implementation.

# Chapter 4

# ANother Tool for Language Recognition (ANTLR)

ANTLR (ANother Tool for Language Recognition) [51] is a widely used parser generator for building languages, tools, and frameworks. It generates lexers and parsers from grammar specifications, enabling efficient implementation of domain-specific languages (DSLs), data formats, and programming language processors.

ANTLR grammars are typically divided into two parts: **a lexer**, which converts raw input into a stream of tokens, and **a parser**, which interprets that token stream according to a specified grammar. These grammars are written in a syntax similar to Backus-Naur Form (BNF), extended with annotations and directives to support advanced features such as precedence handling, semantic predicates, and embedded actions. ANTLR supports multiple target languages (including Java, Python, C#, etc.), meaning it can generate parser and lexer implementations directly in these languages. This allows developers to write a grammar once and reuse it across diverse programming environments.

## 4.1 Parser generation

ANTLR generates parsers by transforming grammar specifications into recursive descent parsers [34] based on its *adaptive LL(*)* algorithm [35]. This approach combines the simplicity of top-down parsing with dynamic lookahead capabilities, enabling efficient handling of complex and ambiguous grammars without requiring manual lookahead hints. The generated parser works alongside a lexer to tokenize input streams and construct parse trees, while also supporting error recovery and semantic predicates for context-sensitive language features. This makes ANTLR particularly effective for real-world languages with intricate syntactic and semantic requirements without being too hard to write and maintain.

## 4.2 Actions

ANTLR *Embedded actions*, sometimes just called *Embedded code*, enable grammar authors to inject target-language code directly into grammar rules, executing custom logic during specific phases of lexing and parsing. This enables precise control over syntax processing by enabling immediate responses to recognized patterns. Using actions, we can dynamically build abstract syntax trees, enforce semantic rules, or gather contextual

metadata directly during the parsing process. A particularly notable application of embedded actions is their use in supporting Python-style indentation, which requires better semantic understanding and custom state management to accurately implement the logic for handling indentations and dedentations. This can be observed in the implementation of Python 3's grammar, originally developed by Bart Kiers and later incorporated into the ANTLR team's official grammars-v4 repository—a community-maintained collection of ANTLR grammars contributed by authors worldwide. In this grammar, embedded actions are used to ensure the correct handling of Python's indentation rules [4]. However, this flexibility comes at the cost of portability: grammars become tightly coupled to both the implementation language and specific application logic.

ANTLR also has listener and visitor mechanisms as an alternative that offers a cleaner separation between grammar specifications and implementation logic [35].

# Chapter 5

# Prolog

Prolog is a high-level programming language based on the paradigm of *logic programming*. Unlike imperative languages such as Java or Python, where programs are expressed as explicit sequences of instructions, Prolog programs are defined as sets of logical relationships. In Prolog, the programmer specifies *what* constitutes a solution rather than explicitly detailing *how* to compute it. This declarative approach allows Prolog to automatically handle the underlying search and inference processes, making it particularly suitable for tasks involving symbolic reasoning, pattern matching, and the analysis of complex structures.

In the context of programming language tooling, Prolog's strengths become especially relevant. Tasks such as semantic analysis, symbol resolution, and reference finding inherently involve reasoning about relationships and structures within code, making Prolog a natural fit for these types of tasks.

## 5.1 Facts, Rules, and Queries

The foundation of any Prolog program rests on three primary elements:

**Facts** Facts are fundamental assertions of truth within the program's domain. They represent direct, unconditional statements about objects and their relationships. For instance, `parent(john, mary).` declares that John is a parent of Mary. These facts form the base knowledge from which Prolog can reason.

**Rules** Rules extend this knowledge by defining conditional relationships. A rule specifies how new information can be inferred if certain conditions, expressed as a body of goals, are met. The syntax `grandparent(X, Y) :- parent(X, Z), parent(Z, Y).` illustrates this: $X$ is a grandparent of $Y$ if $X$ is a parent of some $Z$, and that $Z$ is a parent of $Y$. Rules enable Prolog to deduce more complex relationships from simpler ones.

**Queries** Queries are the means by which users interact with a Prolog program, asking questions about the defined facts and rules. A query like `?- grandparent(john, alice).` asks the system to determine if the stated relationship holds. Prolog attempts to satisfy the query by applying its inference mechanisms. Queries can also contain variables, such as `?- grandparent(john, X).`, prompting Prolog to find all possible values for $X$ that make the statement true.

## 5.2 Unification and Search

Prolog's ability to answer queries and derive inferences relies on powerful built-in mechanisms:

**Unification** At the heart of Prolog's operation is unification, a sophisticated pattern-matching process. When evaluating a rule or query, Prolog attempts to make terms identical by systematically binding variables to values. For example, if matching `parent(john, Child)` against the fact `parent(john, mary)`., `Child` is unified with `mary`. This mechanism is crucial for selecting relevant facts and applying rules based on structural similarity.

**Backtracking** Prolog employs an automatic search strategy known as backtracking. When attempting to satisfy a goal, if one path of reasoning leads to a dead end or if multiple solutions are possible, Prolog automatically retracts its steps (backtracks) to explore alternative paths or variable bindings. This built-in search capability allows Prolog to systematically explore the solution space defined by the facts and rules without requiring the programmer to implement explicit search algorithms.

## 5.3 Data Representation and Manipulation

Prolog represents data using atoms, variables, and compound terms (structures):

- **Atoms** are constants representing specific entities or values, such as `john`, `mary`, or `blue`.
- **Variables** are placeholders for unknown values, typically starting with an uppercase letter or underscore, such as `X`, `Child`, or `_`.
- **Compound terms** (structures) represent complex data composed of a functor and arguments, such as `date(2025, 5, 7)` or `point(X, Y)`.

In addition to these primitives, the language also provides built-in support for lists, enabling convenient handling of sequences.

## 5.4 Relevance for Semantic Code Analysis

Prolog's declarative nature, combined with its powerful built-in mechanisms such as unification, recursion, and backtracking, makes it particularly well-suited for semantic analysis tasks in programming languages. Source code is often represented as structured data such as CSTs or ASTs. Prolog naturally excels at handling such hierarchical and relational structures. For instance, a CST can be represented as a set of Prolog facts, each describing nodes, their types, values, and relationships to other nodes. Semantic rules can then be defined as Prolog rules that interpret these facts, extracting meaningful information such as symbol declarations, references, and scopes. This approach aligns well with the principles of *attribute grammars* [21], a formal method for specifying semantics by defining attributes for grammar symbols and rules for computing these attribute values based on a node's context in the AST. Prolog rules can naturally express these attribute computations, allowing for semantic analysis like symbol resolution and type checking by propagating information up (synthesized attributes) or down (inherited attributes) the AST.

# Chapter 6

# Related work

## 6.1 Grammars and Their Inference

The automated inference of grammars from examples, a field known as grammatical inference or grammar induction, has a significant history within computer science and has seen various applications in software engineering. As surveyed by Stevenson and Cordy [48], the core aim of grammatical inference is to automatically derive a formal grammar that describes an unknown language, given a set of sample strings (or programs) from that language. Traditional approaches have explored diverse learning models and techniques, ranging from interactive systems requiring human guidance to evolutionary algorithms, often targeting the inference of *context-free grammars* (CFGs) for general-purpose or domain-specific languages. Other notable techniques include the Nevill-Manning–Witten algorithm (Sequitur) [28], which infers a CFG by identifying and replacing repeated sequences within a single input string to form hierarchical rules, primarily aimed at compression and structure discovery. These efforts have laid the groundwork for automatically understanding the possibilities and inherent complexities of learning syntactic structures.

More recent advancements in grammatical inference, particularly relevant to practical software engineering scenarios, have focused on learning CFGs from positive examples using a Boolean-valued oracle. This oracle can determine if a given string is part of the target language, a role often fulfilled by an existing parser or validator. *GLADE* [5] represents a notable system in this paradigm, learning CFGs by first inferring regular expressions for segments of the input examples and subsequently merging these to form more general grammar rules. Building upon this, *ARVADA* [22] introduces an algorithmic approach that learns CFGs by iteratively building and restructuring parse trees from positive examples. ARVADA employs “bubbling“ operations to introduce new non-terminals for sequences of sibling nodes and “merging“ operations to generalize by equating non-terminals, guided by oracle queries to ensure validity. Kulkarni, Lemieux, and Sen [22] demonstrates that ARVADA can significantly improve recall and F1-score over GLADE, particularly for grammars with highly recursive structures, such as those often found in programming languages. Subsequently, *Kedavra* [23] proposed an incremental grammar inference method that first decomposes example strings into smaller, simpler units and then progressively builds the grammar, reporting improvements in grammar quality, readability, and inference speed over systems like Arvada.

These oracle-based methods, such as GLADE, ARVADA, and Kedavra, operate by learning from positive examples alongside a Boolean-valued oracle. This oracle functions

as a black-box verifier, capable of answering membership queries—that is, determining if a given candidate string is a valid member of the target language. In practical software engineering contexts, this oracle is typically an existing, trusted parser or compiler for the inferred language, which these algorithms query to validate proposed grammar generalizations and prevent over-generalization.

These oracle-based algorithmic approaches share a conceptual similarity with uLangs, where the ANTLR toolchain implicitly acts as a powerful verifier (oracle) for candidate grammars. However, uLangs diverges by employing LLMs as the primary engine for generating grammar hypotheses (in the specific ANTLR v4 format), which are then iteratively refined based on the formal feedback from this verifier, rather than relying on explicit algorithmic construction rules.

Recent work has also explored using LLMs in conjunction with formal grammars to improve the generation of structured outputs. For instance, Wang et al. [53] introduce "grammar prompting," where providing a specialized Backus–Naur Form (BNF) grammar alongside examples helps LLMs generate more accurate DSL programs. Their approach focuses on guiding the LLM to produce instances of a language, whereas uLangs aims to synthesize the definition of the language's syntax (via ANTLR grammars) and semantics for tooling purposes.

**Very recent work by Torkamani [52] introduces Kajal, a system that also leverages Large Language Models for automatically inferring grammars from DSL code snippets.** Notably, Kajal shares several foundational principles with uLangs: it employs prompt engineering and few-shot learning, and critically, it utilizes an iterative generate-verify-repair loop. In Kajal's process, an LLM generates a candidate grammar, which is then tested by attempting to parse the input code snippet using the Lark parsing library. If parsing fails, the error message from the parser is fed back to the LLM as context for refinement, a mechanism for iterative improvement that mirrors the feedback strategy in uLangs. The emergence of systems like Kajal, which also successfully use this LLM-driven, error-feedback-based iterative cycle for grammar induction, lends empirical support to the idea that even if the LLM's internal generation process can be considered heuristic or "opaque," the overall synthesis approach can achieve empirically reliable and progressively more accurate results. Both systems demonstrate that by grounding the LLM's generative capabilities with concrete verification against a parser and iteratively refining based on explicit error signals, the inherent nondeterminism or potential for initial inaccuracies in LLM outputs can be effectively managed and steered towards correct grammar synthesis.

However, while both approaches share this core iterative, feedback-driven philosophy, key distinctions exist in their goals and scope. Kajal primarily focuses on extracting a parsable grammar (evaluated on parsing accuracy) suitable for the Lark parsing ecosystem. In contrast, uLangs aims to synthesize a more comprehensive set of artifacts for language server functionality, explicitly targeting the generation of ANTLR v4 lexer and parser specifications. This choice of ANTLR is strategic, leveraging its mature toolchain for robust parsing and detailed error diagnostics, which are integral to uLangs' distinct feedback mechanism. Furthermore, a significant differentiator is that uLangs extends beyond syntactic analysis to also synthesize a Prolog-based semantic adapter, a crucial element for enabling code intelligence features in an LSP server. Kajal's scope, as presented, is centered on grammar extraction. Consequently, the evaluation methodologies also differ, with uLangs assessing the functional correctness of LSP features derived from its synthesized components, rather than solely grammar parsing accuracy.

### 6.1.1 Critique

Grammatical inference (GI) has a long history of algorithmic approaches designed to learn grammars from examples automatically. Recent systems such as GLADE, ARVADA, and Kedavra represent significant advancements, particularly in learning CFGs from positive examples, often guided by an oracle that validates string membership. While these methods have demonstrated effectiveness in various contexts, directly applying them as standalone solutions to uLangs' specific goal—automating the synthesis of practical language server components from real-world code examples—presents several challenges.

First, the grammars produced by algorithmic GI methods are typically abstract CFGs, often represented in generic formats such as BNF. While theoretically sound, these grammars are not immediately usable as practical parsing tools within IDEs or language servers. Converting such abstract grammars into robust, executable parsers and lexers suitable for real-world tooling is a non-trivial engineering task.

Second, algorithmic GI methods often rely heavily on structural generalization heuristics, such as state merging, rule bubbling, or incremental decomposition. These heuristics can struggle when faced with the complexity and diversity inherent in real-world programming languages. Such languages frequently include large token sets, nuanced operator precedence rules, idiomatic constructs, and context-sensitive features. Without explicit domain knowledge or careful curation of examples, algorithmic GI methods may either overfit to provided examples or produce overly generalized grammars that fail to capture important language nuances.

Third, many existing GI approaches assume the availability of an oracle—typically a pre-existing parser or validator for the target language—that can provide simple yes/no feedback on whether a given string belongs to the language. In uLangs' scenario, however, such an oracle is not required. Indeed, the very goal of uLangs is to synthesize the components that would form part of such a system. Moreover, even if such an oracle existed, its binary feedback would lack the detailed diagnostic information necessary to guide fine-grained grammar refinements effectively.

Given these challenges, uLangs adopts a different approach, leveraging LLMs as flexible hypothesis generators for syntactic structure. LLMs, trained on vast code and natural language corpora, can produce initial grammar hypotheses that implicitly incorporate common programming language patterns and idioms. This capability potentially allows LLMs to generate grammars that generalize better from limited examples, reducing the risk of immediate overfitting or overly narrow grammars.

However, uLangs requires a concrete, verifiable grammar formalism to utilize these LLM-generated grammar hypotheses practically. Here, the choice of ANTLR v4 emerges as particularly beneficial. ANTLR provides a mature, widely used parsing framework that naturally separates lexical and syntactic concerns into distinct lexer and parser specifications. Its grammar files are relatively human-readable and align closely with how developers typically conceptualize language syntax. Crucially, ANTLR's robust toolchain can formally verify the syntactic correctness of candidate grammars and generate parsers that can be tested directly against provided code examples. When parsing fails, ANTLR produces detailed, actionable error messages, enabling precise feedback for iterative refinement.

Contemporary research underscores the viability of using LLMs within an iterative, feedback-driven loop for grammar synthesis. For instance, Kajal also uses an LLM with a parser-feedback loop (using the Lark parsing library) to infer grammars from DSL code snippets, demonstrating that such an approach can achieve empirical success. This

supports the premise that even if the LLM's generation process is heuristic, coupling it with a concrete verification step and iterative refinement based on parser errors can lead to reliable outcomes. The detailed error messages and diagnostics produced by ANTLR during these steps serve as actionable feedback signals for uLangs' *generate-verify-repair* loop (Sections 8.2 and 9.4).

This combination of LLM-driven grammar synthesis and rigorous verification through ANTLR's robust toolchain forms the core of uLangs' methodology. It leverages the LLM's capacity to generalize syntactic patterns from examples and its implicit knowledge of programming language constructs. It also systematically mitigates errors through iterative feedback provided by ANTLR's precise diagnostics. While LLMs present their challenges, the structured feedback loop within uLangs is designed to minimize these, steering the synthesis process toward correctness and practical usability for LSP component creation.

While uLangs currently relies on LLMs for grammar hypothesis generation, it is worth noting that traditional algorithmic grammatical inference techniques could potentially complement this process. For example, methods like GLADE or ARVADA might be used to suggest candidate grammar fragments, identify recurring patterns, or validate portions of the grammar synthesized by the LLM. Exploring such hybrid approaches—where algorithmic GI methods and LLMs inform or refine each other—could further improve the robustness and generalizability of automated grammar synthesis in future iterations of uLangs.

## 6.2 Semantic Understanding of Code

Semantic analysis is crucial for advanced language tooling, but creating semantic analyzers has required manual effort and domain expertise. Recent research has explored logic programming and Large Language Models to automate and simplify this process.

Logic programming languages like Datalog have proven effective for expressing complex semantic analyses. For instance, Soufflé [20] allows analysts to write concise Datalog rules, which it then compiles into efficient C++ analyzers capable of handling large codebases like OpenJDK. While Soufflé demonstrates the scalability and practicality of logic-based analysis, experts still need to craft the core logic rules manually.

More recently, researchers have begun leveraging LLMs to automate parts of semantic analysis. Using an LLM, KNighter [58] synthesizes static analysis checkers directly from historical bug patches. It uses an iterative refinement loop, validating and improving the generated checkers based on their accuracy in detecting known bugs. Similarly, LLMSA [54] introduces a neuro-symbolic approach, combining user-defined Datalog policies with LLM-generated semantic relations. LLMSA carefully structures the LLM's tasks to reduce hallucinations, demonstrating how LLMs can effectively augment symbolic analysis frameworks.

uLangs builds upon these insights, combining the strengths of logic programming and LLM-driven synthesis. Like Soufflé, uLangs uses a logic programming language (Prolog) to represent semantic analysis rules. However, uLangs employs an LLM to automatically synthesize these Prolog rules instead of requiring manual rule creation. Given linearized Abstract Syntax Tree (AST) facts from the ANTLR parser and a target JSON schema representing semantic information (such as symbol definitions and references), the LLM generates candidate Prolog code. This code is then rigorously verified through Prolog execution, schema validation, and an LLM-as-a-judge scoring mechanism, with feedback

guiding iterative improvements. Thus, uLangs shares the iterative, feedback-driven philosophy of KNighter and LLMSA but applies it to synthesizing a general-purpose semantic adapter for language server features.

## 6.3 Gaps in Automated Code Intelligence

The preceding sections have reviewed considerable research in grammatical inference (Section 6.1) and various approaches to semantic analysis (Section 6.2). Despite these advancements, our literature review reveals a significant gap: there is very limited published research addressing the integrated and automated synthesis of both the syntactic and semantic foundations required for general-purpose code intelligence, directly from a corpus of code examples. Furthermore, our search did not uncover other automated approaches aiming to comprehensively synthesize unified code intelligence components, even beyond example-driven methods.

Grammatical inference methods, as discussed, offer a rich history of techniques for learning syntactic rules, but they typically do not extend to semantic interpretation. This leaves a critical aspect of code understanding unaddressed if one were to rely solely on these techniques for building comprehensive code intelligence.

Conversely, while powerful, existing semantic analysis approaches present their limitations when considered for uLangs' goal of automated synthesis from examples. Traditional static analysis frameworks and even declarative logic programming systems like Soufflé depend on human experts to meticulously define the semantic rules and analysis logic. This manual crafting is knowledge-intensive and language-specific. Even recent LLM-augmented semantic tools, such as KNighter for bug checker synthesis or LLMSA (Wang et al., 2024) for populating user-defined Datalog policies, are often tailored to specific tasks or assume a pre-existing structural understanding of the code, rather than synthesizing a general-purpose semantic model from the ground up purely from code examples. Furthermore, adapting these semantic analyzers to new or niche languages remains a considerable undertaking, and most inherently assume the availability of prior syntactic preprocessing.

More broadly, both academic research and commercial efforts in AI-driven developer tooling have predominantly focused on a relatively uniform set of directions: code generation assistants (e.g., GitHub Copilot [17]), conversational interfaces for querying codebases, and task-specific automation within well-supported languages. While these tools demonstrate impressive capabilities, they generally leverage existing language support or extensive pre-training and do not typically attempt to synthesize the underlying, reusable syntactic and semantic understanding of a language in a unified manner. Our extensive search indicates that integrated approaches—those aiming to automatically create both parsing and general semantic analysis capabilities from code examples—are not a primary focus of current published research.

**This gap highlights a clear opportunity:** the need for a systematic approach to automatically synthesize foundational code intelligence components, such as parsers and semantic analyzers, directly from code. Addressing this gap would significantly reduce the manual effort required to provide rich code intelligence, especially for new, niche, or rapidly evolving languages lacking robust tooling. Furthermore, the successful application of Large Language Models in synthesizing complex artifacts in related domains, such as grammars for DSLs (Torkamani [52]), static analysis checkers (Yang et al. [58]), and guiding semantic interpretation (Wang et al. [54]), suggests that an LLM-driven approach, coupled with robust verification, is a promising avenue for tackling

this challenge. This is precisely the opportunity and methodology that uLangs aims to explore.

# Part II

# Implementation

# Chapter 7

# System Overview

This chapter provides an overview of the uLangs system, describing its architecture and how its components interact as a unified toolkit for language server generation. The design is inherently modular: it comprises a syntax parsing module, a semantic analysis module, and a universal query engine, which can be used as the core engine in an LSP server; these modules are made to work end-to-end. The toolkit leverages established frameworks (ANTLR for parsing, SWI-Prolog for logic reasoning), which serve as deterministic validation methods within our LLM-based pipeline, ensuring the automated synthesis process remains robust and verifiable.

The implementation of the uLangs toolkit, detailing its core components and their design choices, is elaborated across the following chapters:

- **Chapter 7 (System Overview)**, the current chapter, introduces the uLangs toolkit's architecture, detailing its modular components (syntax parsing, semantic analysis, query engine), foundational technologies, and the core concept of the Language Description Specification.
- **Chapter 8 (Syntactic analysis and synthesis)** focuses on the automated generation of ANTLR grammars from code examples using LLMs, detailing the iterative feedback loop where ANTLR's diagnostics guide grammar refinement.
- **Chapter 9 (Semantic analysis and synthesis** describes the transformation of syntax trees into Prolog facts and the LLM-driven synthesis of a Prolog semantic adapter to extract symbol information, refined via schema validation and LLM-as-a-judge scoring.
- **Chapter 10 (Universal LSP Server)** demonstrates the practical application of the synthesized artifacts by presenting a proof-of-concept LSP server that uses the generated components to provide code intelligence.
- **Chapter 11 (Key Algorithms)** offers an in-depth look at the core generate-verify-repair cycle, including LLM dialogue management, feedback mechanisms, context compression, and input file handling strategies.
- **Chapter 12 (Challenges and Trade-offs)** discusses critical design choices, technical challenges, and trade-offs made during development, such as technology selection and LLM considerations.

This overview sets the stage for the following implementation chapters, each of which delves into a specific subsystem or algorithm in detail, illustrating how the codebase collectively tackles the challenges of automating language server construction.

## 7.1 Unified toolkit

Universal Language Server Toolkit, or *uLangs* for short, has two major components, the **Grammar component** and the **Semantic component**. Each of these has a **synthesis** and an **inference** (or query) mode. The two major components are designed to work seamlessly together as the Semantic component is heavily coupled to the Grammar component and relies on its parsed output. The output from the semantic component can almost directly be used as the LSP method response. This is what is meant by an end-to-end system providing code intelligence.

This toolkit was designed specifically to automate the creation of LSP language servers, particularly targeting languages where existing tooling is insufficient, such as uncommon DSLs, though its applicability is not limited to these. A critical requirement for achieving this automation was ensuring that the synthesized artifacts could be queried easily and directly from an LSP implementation or an IDE. Equally important was that the output from these queries had to be structured, meaningful, and require minimal additional processing before being directly usable as standard LSP method responses. Figure 7.1 illustrates how data flows through the uLangs system when queried in inference mode. By combining the synthesized LDS (discussed in Section 7.2.1) with specific main semantic queries (Section 9.2), uLangs enables direct querying of arbitrary source code to produce precisely the structured semantic information needed to implement LSP features.

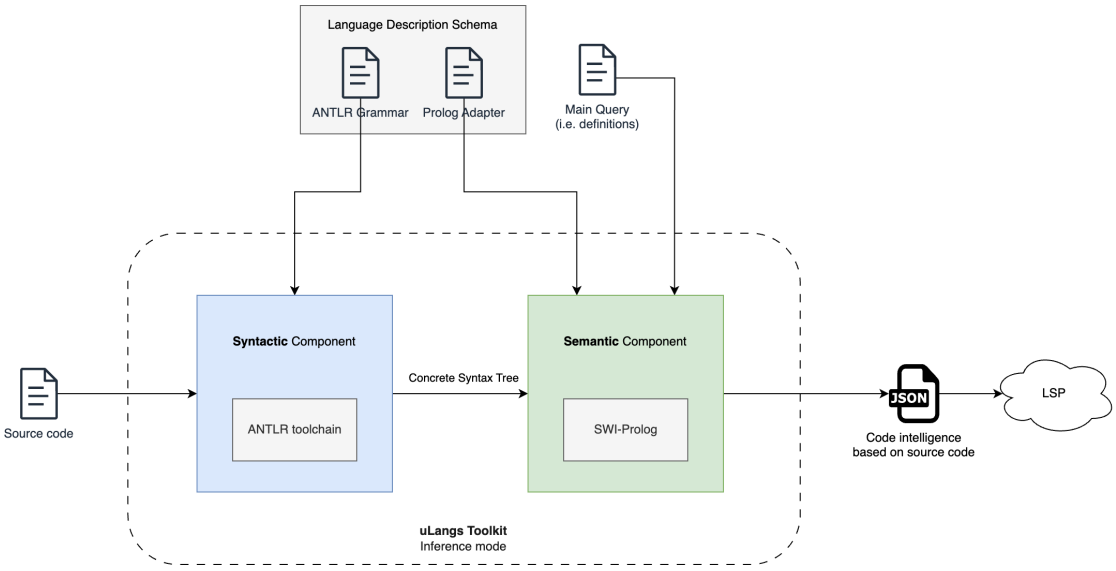



Figure 7.1: This diagram illustrates the data flow in uLangs inference mode, showing how language specifications and query definitions transform source code through syntactic parsing and semantic analysis into structured meaningful code intelligence.

A key aspect of the synthesis mode in this system is its *feedback loop*, both components are not generated in isolation but are refined through iterative verification against a set of verification and feedback systems, achieving a higher degree of quality and correctness.

**Verification** In the context of this thesis, Verification is the process of evaluating a candidate artifact (grammar or adapter) against specific code examples using predefined checks. For grammars, this involves checking successful parsing via ANTLR

(see Section 8.2). For adapters, it consists of checking Prolog execution, schema adherence of the output JSON, and potentially semantic scoring (see Section 9.4). The outcome—success or failure with specific diagnostics—directly informs the feedback signal used in the iterative repair loop.

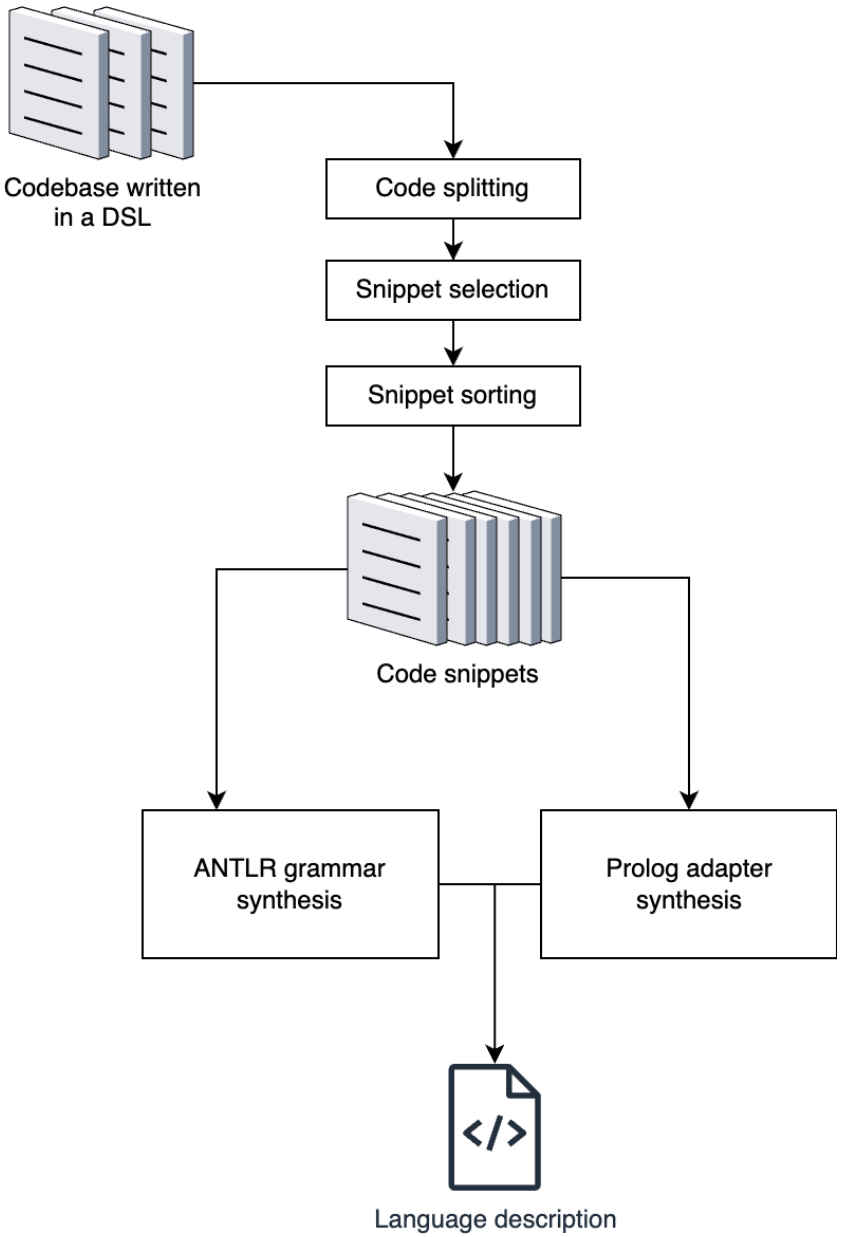



Figure 7.2: Snippet-driven synthesis in uLangs. The system processes a DSL codebase by splitting, selecting, and sorting snippets. It then generates an ANTLR grammar and a Prolog adapter, producing a structured language description.

## 7.2 Designing a machine-readable language description schema

At the outset of the uLangs project, we recognized the need for a concise and precise way to represent arbitrary programming languages' syntactic and semantic structure. Initially, we considered designing a fully custom specification format from scratch, tentatively calling it a *Language Description Specification*, that could capture all necessary language details in a single, structured representation. At this early stage, we did not yet know precisely what artifacts our synthesis pipeline would need to generate. Still, we understood clearly that a strict and well-defined language description would be essential for synthesis and runtime components.

However, as we explored existing technologies, we realized we might not need to invent an entirely new specification format. Instead, we could leverage established tools that already provide concise and formal ways to describe syntax and semantics. Specifically, we identified ANTLR grammars as a natural fit for capturing syntactic structure, and Prolog as a powerful and declarative way to represent semantic rules. By combining these two existing technologies, we could achieve the goals we initially envisioned for the LDS without reinventing the wheel.

This realization significantly shaped the direction of the project. It clarified exactly what artifacts our synthesis pipeline needed to produce: ANTLR grammar files and Prolog adapters. It provided a clear target for the LLM-driven synthesis process. Adopting ANTLR and Prolog as the core components of our language description, it established a robust foundation for the entire uLangs toolkit. We retained the term *Language Description Specification* to refer to this combined specification, reflecting our original goal of a unified, structured representation, now realized through the integration of existing, proven technologies.

### 7.2.1 The Language Description Specification (LDS)

Having established the foundational technologies for this specification, a more concrete definition is possible. The primary purpose of the LDS is to serve as the structured, machine-readable output of the uLangs synthesis pipeline. It encapsulates all the synthesized knowledge about a target language, both syntactic and semantic, in a format directly consumable by runtime components, such as the query engine and the Universal LSP Server. In other words, the LDS acts as the critical bridge between the automated synthesis process and the practical implementation of language tooling.

In its current conceptual form, the LDS produced by uLangs consists of three main artifacts:

1. An **ANTLR lexer** grammar file (Lexer.g4), defining the tokenization rules of the language.

2. An **ANTLR parser** grammar file (Parser.g4), specifying the syntactic rules for parsing tokens into parse trees.

3. A **Prolog adapter** file (Adapter.pl), containing synthesized semantic rules that transform parse trees into ASTs and extract symbol information.

These three files collectively represent the complete synthesized description of the target language. The choice of ANTLR and Prolog as the underlying technologies for the LDS was deliberate, driven by their suitability for automated synthesis, clear

parse tree representation, and straightforward integration with semantic reasoning. (See Sections 12.1 and 12.2 for detailed discussions of these choices.)

Currently, the LDS is conceptually minimal: users point uLangs (in inference mode) at these three files. However, a slightly more user-friendly approach would be to package these artifacts into a single, self-contained file, such as a compressed archive (e.g., a ZIP file) or a structured manifest file (e.g., JSON or YAML), that explicitly references the grammar and adapter files and includes relevant metadata. This packaging would simplify distribution, versioning, and integration with downstream tooling.

The design of the LDS follows several guiding principles. First, it must be *complete*, ensuring that all necessary information for runtime tooling is explicitly captured. Second, it should be *consistent*, providing a predictable structure across different synthesis runs and languages. Finally, it should remain *simple and modular*, facilitating both automated generation by uLangs for any arbitrary programming language and straightforward consumption by runtime components.

Looking forward, a complete and robust version of uLangs would require extending the LDS to support inter-file dependencies and project-level semantics. As discussed in Section 9.5, languages often rely on relationships across multiple files, such as imports, modules, or inheritance hierarchies. To accommodate these scenarios, the LDS would likely need additional components, such as:

- A metadata file specifying project-level information, including file extensions, language versioning, and configuration details.
- A dedicated loading semantics file or "Project Adapter," defining how multiple files relate to each other, how dependencies are resolved, and how the semantic context is established across files.

# Chapter 8

# Syntactic analysis and synthesis

Syntactic analysis forms the first pillar of the uLangs toolkit, which is responsible for understanding the syntax and structure of the target language through constructing grammars that can successfully parse the user's input samples. This chapter details how uLangs synthesizes an ANTLR grammar for a given language from the code examples. The system can start from an initial grammar (if available) or generate one from scratch using an LLM primed with grammar induction patterns. As the grammar is a critical piece for representing a language, we introduce a feedback signal mechanism: the generated grammar is iteratively tested by parsing known code snippets or test inputs, and any parse failures or ambiguities are fed back into the synthesis loop for refinement until satisfactory results emerge. By automating grammar creation and validation, uLangs significantly lowers the barrier to establishing a correct parser, which is the foundation for all further semantic analysis.

## 8.1 Synthesizing ANTLR grammars

The first step in the uLangs synthesis pipeline is generating an initial ANTLR grammar that accurately captures the syntax of the target language. This grammar is foundational as later semantic analysis and tooling depend on its correctness and quality. To achieve this, uLangs leverages the generative capabilities of LLMs, instructing them to infer grammar rules directly from provided code examples.

In most cases, the grammar synthesis process begins entirely from scratch, without any pre-existing grammar definitions. However, uLangs also supports scenarios where users provide an initial or partially complete grammar as a starting point. In these cases, the synthesis process follows the same overall logic, with the key difference being that the provided grammar serves as the initial candidate rather than being generated from example code snippets. The repair and refinement mechanisms remain identical, ensuring that the grammar evolves iteratively regardless of whether it starts from scratch or builds upon an existing definition. While both approaches are supported, the primary use case and most thoroughly explored is synthesizing grammars entirely from scratch using example code snippets.

To guide the LLM effectively, uLangs employs a carefully designed prompting strategy. The prompt explicitly instructs the model on the specifics of ANTLR grammar syntax, including the distinction between lexer rules (uppercase token definitions) and parser rules (lowercase syntactic constructs). It also provides a concise overview of key parsing concepts, such as LL(*) parsing, recursive descent parsing, and ambiguity resolution. Crucially, the prompt clearly defines the task: given a set of code snippets,

the LLM must generate a complete ANTLR grammar consisting of two files, `Lexer.g4` and `Parser.g4`, with a designated start rule named `program`. The prompt further instructs the model to produce "generic" grammars, to promote generalization beyond the provided examples, and to output the complete grammar files in a structured and consistent format.

Currently, during the initial grammar generation step, uLangs simultaneously provides the LLM with all available code snippets. This approach ensures the model has sufficient context to infer general syntactic patterns and produce a comprehensive initial grammar candidate. However, this strategy has practical limitations, particularly when dealing with large codebases or extensive example sets. In such cases, providing all snippets at once may exceed the LLM's context window or introduce unnecessary complexity (as model performance degrades as the context grows [24, 27]). A more scalable approach, discussed further in Section 11.4.3, would involve selecting a diverse and representative subset of snippets that effectively captures the language's syntactic variety without overwhelming the model.

The grammar candidates generated by the LLM typically exhibit minor inaccuracies or oversights, similar to those a human grammar author might initially make. Common issues include incorrect handling of operator precedence, ambiguous rule definitions, or overly simplistic tokenization. However, these initial imperfections are expected and are systematically addressed by the feedback-based repair loop (See section 11.1). Through iterative verification and refinement, the grammar quickly converges toward correctness, leveraging the LLM's ability to incorporate structured feedback and correct its earlier mistakes.

## 8.2 Feedback signal

The grammar generated by the LLM in the previous step is initially only a candidate solution. To ensure correctness, uLangs employs a structured verification process, systematically testing the candidate grammar against the provided code examples. When the grammar fails to parse an example correctly, the system generates a detailed feedback signal, which is the primary driver for iterative refinement and repair.

The verification process is relatively straightforward: uLangs attempts to compile the candidate ANTLR grammar files and then uses the resulting parser to parse each provided code snippet. If the grammar compiles successfully, the system proceeds to parse the code examples one by one. As soon as a parsing failure occurs, the verification stops immediately, and the system collects all errors and warnings produced by ANTLR for that specific failing example. (Section 11.1 describes this verification step's logic in more detail.)

ANTLR errors encountered during verification are captured and structured into a consistent format within uLangs, represented by the `ANTLRError` type. Each error is tagged with metadata indicating its `type`, specifying whether the error originated from the lexer grammar (`LEXER`), the parser grammar (`PARSER`), or if the source file is unclear (`UNKNOWN`). Additionally, the error includes a `source` field, indicating the phase at which the error occurred: either during grammar compilation (`BUILD`) or during the parsing of a specific code example (`RUNTIME`). Each error also contains a descriptive message and, when available, precise location information such as filename, line number, and column number. How these ANTLR errors are generated is discussed in Section 12.1.1.

This structured error information forms the core of the feedback signal provided back to the LLM. By clearly indicating the nature and location of each failure, the feedback

signal enables the LLM to better understand what went wrong with its previous grammar candidate. For example, a typical feedback signal might indicate a parser error such as `mismatched input '}' expecting ';'` at a specific line and column in a given code snippet. This detailed context allows the LLM to reason effectively about the necessary corrections, guiding it toward a more accurate grammar in further iterations.

If the *candidate grammar* successfully parses all provided examples without errors, the feedback signal indicates success, and the grammar is considered stable for the current iteration. Conversely, if a particular code snippet repeatedly fails to parse despite multiple repair attempts, uLangs employs a rollback-and-skip mechanism. In this scenario, the problematic snippet is skipped entirely, and the grammar candidate is reverted to the last known stable checkpoint (See Section 11.2.1).

# Chapter 9

# Semantic analysis and synthesis

With the syntactic foundation established, uLangs transitions to semantic analysis, focusing on capturing the meaning of programs to enable advanced language server features like symbol resolution and reference finding. This is achieved through tree linearization, a process that encodes parse trees into a series of Prolog facts. Each node in the tree is represented declaratively, including its type, value, position, and relationships, creating a *queryable* semantic model. Building on this representation, uLangs defines key language server queries, such as *"find the definition of symbol X"* or *"determine the type of expression Y."* Using a combination of template-based logic and AI-driven refinement, the system generates a Prolog-based adapter tailored to the target language. This adapter maps parse tree patterns to semantic concepts like declarations, references, and symbol relationships, enabling precise answers to these queries. Conceptually, the Prolog-based adapter generated by uLangs functions as a synthesized set of rules for an attribute grammar [21]. It defines how semantic "attributes", such as symbol definitions, reference locations, or scope information, are computed from the linearized CST facts and associated with different parts of the program structure.

## 9.1 Tree linearization

After parsing a code snippet with the ANTLR-generated parser, the result is a Concrete Syntax Tree (CST) represented as a nested object structure in memory. To interface with the Prolog-based semantic layer, this tree must be translated into a sequence of Prolog facts that encode its structure and content. This step is called *tree linearization* in this thesis.

The linearization process traverses the CST and emits facts describing each node's identity, structure, content, and source location. Each node is assigned a unique identifier, which is used to define parent–child relationships through has_child/2 predicates. Additional facts capture the node type, any terminal value (such as identifiers or literals), and the original file position using node_location/5.

For example, consider x + 42; this expression might be linearized into Prolog facts as follows:

```
node(1, 'BinaryExpr').
node(2, 'Identifier').
node(3, 'Operator').
node(4, 'NumberLiteral').
```

```
has_value(2, 'x').
has_value(3, '+').
has_value(4, '42').

has_child(1, 2).
has_child(1, 3).
has_child(1, 4).
```

This representation transforms the hierarchical CST into a declarative knowledge base suitable for symbolic reasoning in Prolog. It makes each element of the tree addressable and traceable, enabling fine-grained semantic queries to be written over the structure.

The linearization step abstracts away language-specific parse tree implementations and provides a uniform interface for the adapter and query logic. This uniformity ensures that all snippets, regardless of language, can be handled consistently in the semantic phase.

Tree linearization serves as the bridge between syntax and semantics. It allows the system to reason about structure using logic rules, while retaining precise connections to the original source. Without this transformation, the semantic layer would be unable to operate effectively on parsed data, and adapter synthesis would lack a stable input format.

## 9.2 Main queries

To bridge the gap between syntactic parsing and functional code intelligence, the system provides a core set of Prolog queries explicitly designed to extract the semantic information required to implement standard LSP features. These queries operate over the CST, represented as Prolog facts, potentially transformed by a language-specific Prolog adapter. Their outputs provide the structured data necessary for enabling features like go-to-definition, finding references, and displaying document symbols within an LSP-compatible editor.

Each query produces structured JSON, validated against a predefined schema to ensure consistency and provide a reliable data format for downstream LSP server implementations.

The three main queries, chosen for their direct relevance to core LSP capabilities, are:

- **printAST:** Produces a simplified AST representation. While primarily used during synthesis for validating the adapter's structural mapping, its output can also aid in debugging or visualizing code structure, potentially supporting custom LSP extensions.

- **definitions:** Extracts declared symbols, references, and their source locations. This query directly provides the data needed for fundamental LSP features such as textDocument/definition (go-to-definition), textDocument/references (find references), and potentially parts of textDocument/documentSymbol. The effectiveness of this query relies on the adapter correctly identifying scopes and name bindings.

- **holotype:** As detailed in Section 9.2.1, this query generates a unified AST and symbol table. It serves as the primary verification target during adapter synthesis and acts as a comprehensive data source for implementing multiple LSP features from a single, consistent representation.

Internally, these queries are modular Prolog files dynamically combined with the CST facts and adapter logic. While crucial for the synthesis feedback loop (where failures guide adapter repair), their ultimate operational purpose is to deliver the processed semantic information in a format readily consumable by the universal LSP server component, thereby enabling code intelligence for the target language.

### 9.2.1 Holotype query

The holotype query is designed to generate a single, unified representation that captures both the essential syntactic structure (AST) and foundational semantic information (symbol table) for a given code snippet, as interpreted by the semantic system and a candidate Prolog adapter. Its primary purpose is to produce the canonical output for the semantic analysis of a snippet. This output serves as the core validation target during the adapter synthesis process, ensuring the adapter correctly models the language's structure and meaning. The term *holotype* is inspired by biological taxonomy, which refers to the single, authoritative specimen chosen to define and represent a newly discovered species formally. In biology, the holotype is the definitive reference point against which all other specimens are compared. Similarly, in uLangs, the holotype query is designed to yield a single, authoritative representation that captures the syntactic and semantic characteristics we care about from a given code snippet. Just as a biological holotype defines the standard for a species, the output of the holotype query is used to compare and set the standard for correctness during adapter synthesis, providing a target for validation and iterative refinement.

Operationally, the system first parses the code snippet using the ANTLR-generated parser into a CST. This CST is then linearized into Prolog facts detailing node identities, structure, content, and source locations. The synthesized Prolog adapter, containing language-specific rules, processes these facts to derive an AST and identify symbol declarations and references. If this is successful (meaning no error and a high score, see Section 9.4), then our candidate adapter is sufficient to run our main queries.

Executing the holotype query applies the adapter's logic to the CST facts, yielding a structured JSON object. This object contains the derived AST and the list of symbols with their locations, which is then validated against a predefined schema for correctness and consistency. As mentioned, these queries are stored as modular Prolog files, making it easy to change or add new queries if new requirements arise (e.g., type inference, hover information, etc.). Therefore, if a new *main query* is added, you can easily extend the *holotype query* to ensure that the semantic information required is identified and is considered by the feedback signal (most importantly, *scoring*).

The significance of the holotype query lies in its role within the synthesis pipeline. A successful generation across all test snippets indicates that the adapter accurately captures the target language's essential structure and semantics according to the system's interpretation. Conversely, failures or schema mismatches provide a direct feedback signal for the iterative repair loop, guiding the refinement of the adapter until it reliably produces the correct, unified holotype output for the provided examples.

## 9.3 Synthesizing an adapter

Once the syntactic structure of the target language has been captured through the synthesized ANTLR grammar, the next critical step is to generate the semantic adapter. This adapter, implemented as Prolog rules, bridges the gap between the CST, represented as linearized Prolog facts, and the structured semantic information required by the main queries, such as the *holotype* query. The adapter embodies the language-specific semantic logic inferred by uLangs, transforming raw syntactic data into meaningful semantic representations like ASTs and symbol tables.

The adapter synthesis process closely mirrors the grammar synthesis workflow described previously. Typically, the adapter is generated entirely from scratch, although users can optionally provide an initial or partially complete adapter as a starting point. Regardless of the starting point, the synthesis employs the same iterative generate–verify–repair loop, progressively refining the adapter based on structured feedback signals.

To guide the model effectively during adapter synthesis, uLangs employs a carefully designed prompting strategy. The prompt clearly defines the task: given a set of linearized CST facts representing a parsed source code file, the LLM must generate Prolog predicates that transform these facts into a structured JSON output conforming precisely to a provided formal JSON schema. This schema, derived from a Zod [60] schema definition, explicitly specifies the required structure and fields of the semantic output for each main query (see Appendix B). By providing the complete formal schema directly in the prompt, uLangs ensures the LLM has an unambiguous target for its semantic transformations.

In addition to the schema and CST facts, the prompt includes targeted guidance to help the LLM produce effective Prolog code. For example, it emphasizes clear domain mappings, suggesting how specific syntactic constructs should map onto semantic concepts. It encourages the LLM to define explicit rules for declarations, such as recognizing top-level structures or function definitions, and to provide a single, generic rule for references, ensuring comprehensive symbol extraction without enumerating every possible reference context. The prompt also advises the LLM to closely align its Prolog rules with the actual node types defined in the parser grammar, ensuring accurate and consistent semantic mappings.

To further support the adapter synthesis, uLangs provides a set of predefined Prolog helper predicates. These helpers simplify common tasks such as basic node type checking, tree traversal, child node retrieval, and location extraction. By prepending these helper predicates to the adapter code generated by the LLM, uLangs reduces the complexity of the synthesis task, allowing the LLM to focus primarily on the language-specific semantic logic rather than low-level Prolog boilerplate.

During synthesis, the LLM generates only the core adapter logic (the Prolog predicates responsible for semantic interpretation) within a clearly defined *writable area*. This generated adapter code is dynamically combined with the linearized CST facts, the helper predicates, and the main query implementations at runtime. Although the adapter is conceptually represented as a standalone file (adapter.pl), it is not physically stored during synthesis. Instead, the complete Prolog program, including the adapter, helpers, CST facts, and queries, is dynamically assembled for verification purposes. Once synthesis is complete and the adapter has been validated, the final adapter code is extracted and provided to the user as a standalone artifact for later use.

## 9.4 Feedback signal

A feedback signal similar to the syntactic phase is employed in the semantic analysis, but with more advanced mechanisms to ensure accuracy, as the results are not as easy to verify. Semantic adapters can fail in several distinct ways, as summarized in Table 9.1. This table outlines the types of failures, the order in which the verification module checks them, and the corresponding feedback signals sent back to the generation and repair module. If a failure is detected at any verification step, subsequent checks are skipped, and all feedback from the current step is accumulated and used by the repair logic. This process is guided by two complementary strategies: error-driven refinement and LLM-based scoring. Errors, such as schema mismatches or unresolved references, trigger targeted adjustments to the adapter's logic, while a language model evaluates the new adapter's outputs, scoring their alignment with the expected behavior to guide further improvements.

| Step | Type of Failure | Feedback Signal |
|---|---|---|
| 1 | Invalid Prolog program | Detailed error messages generated by *swi-prolog*. |
| 2 | Empty or invalid JSON output | A simple message indicating the issue. |
| 3 | Schema mismatches | A simple message along with the expected JSON Schema for correction. |
| 4 | Low *LLM-as-a-judge* scoring | Reasons provided by the LLM-based scoring system. |

Table 9.1: Feedback in the semantic synthesis verification system

**Schema validation** To ensure the semantic adapter's output precisely matches the expected structure, uLangs employs strict schema validation using Zod. Each main query implemented in Prolog has a corresponding Zod schema file, creating a clear one-to-one mapping between the query logic and its expected JSON output. This strict validation step guarantees that downstream components receive exactly the structured data expected, preventing subtle integration issues later in the pipeline. If the adapter's output fails schema validation, the system immediately halts further verification steps and generates a targeted feedback signal. This feedback indicates the mismatch and provides the correct JSON schema as a direct hint to the LLM, guiding it in adjusting the adapter logic accordingly.

**LLM-as-a-judge scorer** While schema validation ensures structural correctness, verifying the semantic accuracy of the adapter's output is inherently more challenging. To address this, uLangs incorporates an *LLM-as-a-judge* scoring mechanism, which evaluates the semantic plausibility of the generated AST and symbol information relative to the original code snippet. This scorer operates independently, maintaining its own dialogue management context to act as an *impartial third-party* evaluator. It assigns a numerical score from 0 to 100 based on how accurately the adapter's output reflects the snippet's semantics, heavily penalizing missing, incorrect, or irrelevant symbols and definitions. The scorer also provides explicit textual feedback detailing specific errors or limitations. Currently, a constant threshold (`MINIMUM_JUDGE_SCORE`) is set at 80%; any adapter output scoring below this threshold triggers a feedback signal, prompting the LLM to regenerate the adapter logic. A caching mechanism has been implemented

to reduce the costs of generating this feedback signal. The system caches the score and *scoring reasons* produced by the LLM judge based on the combination of the example code and the final adapter output. This approach simplifies the generic repair loop module and reduces costs in scenarios where the adapter logic is modified but the resulting output remains unchanged. **An interesting future direction could involve extending this scoring mechanism to evaluate multiple semantic categories separately**, each with its own threshold, enabling more nuanced and targeted feedback during adapter refinement. For example, the scorer could independently assess categories such as completeness of symbol extraction, correctness of AST structure, accuracy of reference resolution, and consistency of scope handling. Each category could have its own configurable threshold, allowing the system to provide more granular and actionable feedback. This approach would help the LLM pinpoint which aspects of the adapter logic need improvement and potentially accelerate convergence towards a solution.

Additionally, the current scoring prompt is specifically optimized for the existing *holotype* query (Section 9.2.1), which combines AST and symbol information. While effective for this particular query, this specialization limits the scorer's flexibility and generalizability to other semantic queries or future extensions. An ideal solution would involve designing a more general scoring prompt or a modular scoring system that dynamically adapts to different queries and semantic outputs. Such a generalized scoring mechanism would allow uLangs to more easily support new semantic queries or language features without requiring extensive prompt redesign or fine-tuning.

To make the judge's scoring mechanism robust and machine-readable, uLangs ensures consistency in score outputs by using the Structured Outputs API feature (see Section 3.5).

## 9.5 Inter-file dependencies

A significant limitation of the current uLangs implementation is its focus on single-file semantic analysis. While effective for capturing *intra-file* structure and basic symbol relationships, real-world programming languages and DSLs frequently rely on dependencies spanning multiple files. Features such as module systems (import, require), inheritance across files, separate compilation units, or project-wide configurations necessitate an understanding of how different source files relate to one another - *inter-file* relationships. Providing robust language tooling, particularly for features like cross-file go-to-definition, accurate project-wide refactoring, or comprehensive error checking, requires reasoning about these inter-file dependencies.

The existing uLangs architecture offers a potential foundation for addressing this challenge. Parsing each file into a CST and subsequently linearizing it into Prolog facts already captures essential metadata, including filenames and precise source locations. Theoretically, the Prolog facts representing multiple CSTs from a project could be loaded into a single reasoning session. Prolog's declarative nature is well-suited to querying relationships across such a combined fact base. However, the current system lacks the crucial semantic layer to interpret these relationships correctly. It doesn't know which files constitute a project, how they depend on each other (e.g., via import statements), or the rules for resolving symbols across these boundaries.

This missing piece is not just a question of loading more data, but of defining semantics for multi-file reasoning. To support this, the system would likely need to introduce a new component, separate from both the grammar and the adapter, that governs how a project is assembled semantically. A dedicated component responsible

for managing project-level context seems necessary to bridge this gap. This component, which we might term a **Project Adapter**, would operate at a higher level than the existing (intra-file) semantic adapter. Its primary role would be to establish the semantic context for multi-file analysis before the detailed per-file analysis occurs.

**Responsibilities of this *Project Adapter* could include:**

1. Identifying the set of relevant source files belonging to a project (potentially also guided by configuration files like package.json, pom.xml, or custom project markers).

2. Parsing or analyzing dependency declarations (e.g., import statements) to build a dependency graph between files.

3. Defining the rules for symbol visibility and resolution across file or module boundaries, specific to the target language.

4. Loading the necessary linearized CSTs into the Prolog environment based on the identified dependencies relevant to a specific query or analysis task.

Introducing such a Project Adapter would require modifications to the existing queries. Currently, the query's scope is confined to a single file's AST and symbol table. An extended version would need to represent symbols originating from other files (e.g., imported functions or classes) and potentially include information about the dependency links themselves within its output schema. This expanded holotype would remain the central piece for validation, but its verification would now encompass the correctness of cross-file semantic interpretation.

Although inter-file dependencies represent a considerable extension beyond the current scope, the declarative, fact-based foundation of uLangs, aided by its modular design, makes it conceptually well-suited for this evolution. By introducing a dedicated Project Adapter layer, extending the main queries and their associated schemas, and potentially refining the synthesis feedback mechanisms, uLangs could be adapted to generate language servers capable of understanding and providing intelligence across entire codebases, significantly broadening their applicability to complex, real-world languages.

# Chapter 10

# Universal LSP Server

The preceding chapters have detailed the synthesis pipeline of uLangs, culminating in the generation of an ANTLR grammar and a Prolog semantic adapter tailored to a specific target language. While these synthesized artifacts encapsulate the core syntactic and semantic knowledge derived from code examples, they require a runtime environment to become practically useful for developers within their IDEs. This chapter introduces the *Universal LSP Server*, a proof-of-concept component designed to demonstrate how uLangs can be integrated into a real-world editor workflow.

To demonstrate this integration, uLangs includes a proof-of-concept implementation of a Universal LSP Server. This component is a runtime environment that utilizes the synthesized grammar and semantic adapter to respond to LSP requests, such as symbol definitions, references, and document structure queries. The primary goal of this demonstration is to show that the synthesized language descriptions are sufficient to support real-world LSP features, rather than to provide a fully polished or production-ready language server. It illustrates how little additional effort is required to leverage the synthesized grammar and semantic adapter to implement standard LSP features.

The key aspect of this Universal LSP Server is the explicit mapping between the core semantic queries synthesized by uLangs and standard LSP features. For instance:

- Parse errors detected by ANTLR are directly translated into *textDocument/publishDiagnostics* notifications.
- The output of the *definitions* query, containing symbol declarations and references, forms the basis for responding to *textDocument/definition* (Go to Definition) and *textDocument/references* (Find References) requests.
- The AST and symbol information derived from the holotype query can be used to populate *textDocument/documentSymbol* (Outline view).

With only a few lines of code per LSP method in the extension codebase, the system can already provide meaningful code intelligence features (such as symbol definitions). The core logic - parsing source files, tree linearization, and executing semantic queries - is entirely handled by the synthesized *language description artifacts* and the uLangs query engine. Thus, the extension remains lightweight, primarily serving as a thin communication layer between the IDE and the uLangs core. This extension communicates with the uLangs core system through subprocess calls, invoking the synthesized queries via a simple command-line interface. Input parameters, such as the source code file and cursor position, are passed as command-line arguments, and structured JSON responses are returned via standard output.

However, it is essential to note that this proof-of-concept implementation is currently limited and somewhat unreliable due to technical challenges. Specifically, integrating multiple build systems and dependencies within a single Node.js environment has introduced complexity and instability. The nested codebase structure, mixing TypeScript and Node.js components, has led to frequent build and runtime errors, making the current extension fragile and challenging to maintain. As a result, the provided VSCode extension should be viewed strictly as a demonstration of feasibility rather than a robust, production-ready solution.

Despite these technical limitations, the key takeaway remains clear: all the essential elements required to build a fully functional LSP server are already present within uLangs. The synthesized grammar and semantic adapter and the query engine provide the critical semantic information needed to implement standard LSP features. Future work could focus on resolving the current technical integration challenges and extending its supported LSP methods, resulting in a more stable and user-friendly implementation.

# Chapter 11

# Key Algorithms

The previous chapters have described the overall architecture of uLangs, its theoretical foundations, and some of the technical details of its core components. However, at the heart of this system lies a set of key algorithms that enable the automated synthesis of grammars and semantic adapters.

Central to uLangs' approach is an iterative refinement process, where candidate solutions are repeatedly generated, evaluated, and improved. This iterative loop leverages the generative capabilities of large language models, guided by structured feedback signals derived from strict verification methods. Rather than relying on a single-shot generation, uLangs incrementally refines its generated parts, progressively converging toward grammars and adapters that accurately capture the target language's syntax and semantics.

The effectiveness of the core **generate-and-repair** loop hinges on our ability to properly manage the interactions with the model. By carefully structuring the dialogue with the model, maintaining context, providing targeted feedback, and strategically compressing inputs, uLangs ensures efficient and reliable synthesis, even within the constraints of token limits and computational resources. Additionally, the system employs heuristics to intelligently prioritize input examples, further enhancing the efficiency and quality of the generated solutions.

Real-world input codebases often contain imperfections or syntactic complexities that challenge automated synthesis. To address this, uLangs incorporates mechanisms to handle problematic inputs gracefully, ensuring the synthesis process remains robust and stable. These strategies allow the system to focus on the most informative and reliable examples, avoiding pitfalls that could degrade the quality of the synthesized grammars and adapters.

The following sections provide a more detailed examination of these core algorithms, examining the specific methods and design choices that allow uLangs to automate the creation of language tools efficiently.

## 11.1 Automated generation with Verifier

At the core of uLangs' synthesis capability is an automated algorithm that iteratively generates and refines grammars and semantic adapters. This process, illustrated in Figure 11.1, follows a structured *generate–verify–repair* cycle. Given a set of input examples, the system first prompts an LLM to produce an initial candidate artifact, either an ANTLR grammar or a Prolog semantic adapter. This candidate is then evaluated by a task-specific verifier tailored to the artifact type.

The verifier systematically tests the candidate against each provided example. If the candidate passes verification for all examples, it is accepted as a final solution. However, if verification fails at any point, the verifier generates detailed diagnostics describing the failure. These diagnostics form a structured feedback signal, guiding the LLM to produce an improved candidate in the next iteration, a step we refer to as a *repair*.

This iterative refinement continues until either a candidate successfully passes all verification checks or a predefined iteration limit is reached. Suppose the iteration limit is exceeded for a particular example. In that case, the system employs a rollback-and-skip strategy (see Section 11.2.1), ensuring that synthesis remains robust even when encountering challenging or unparseable inputs.

### 11.1.1 Dialogue management

When communicating with our LLM model to synthesize either ANTLR grammar or the Prolog adapter code, we frame the generation process as a conversation between a user trying to create some code and an assistant. The message array is seeded with a prompt that gives context on its goal as an assistant and useful hints to solve it efficiently. Each back-and-forth message is appended to this message list until a solution is found. This approach is particularly helpful when a candidate needs repair, which is very common when generating a solution. This allows the model to better understand how to ultimately write a solution, as it can *try* different approaches and is given feedback - a kind of interactive, or *agentic*, in-context learning mechanism. This iterative approach helps mitigate the limitations of the initial prompt, which is unlikely to be fully optimal. We have observed that models frequently make some common mistakes early in the conversation, but quickly correct and avoid repeating these mistakes after receiving guidance through our feedback signals. This interactive feedback loop allows models, particularly reasoning models, to incrementally refine their approach, effectively "learning" from previous interactions and ultimately significantly benefiting from the conversational nature of the dialogue.

## 11.2 Feedback-based repair loop

The initial grammars and semantic adapters generated by the LLM represent only the first step in the synthesis process. These initial candidates, while often promising, frequently contain inaccuracies, fail to handle specific edge cases, or do not fully conform to the required output specifications (such as the target JSON schema for adapters). Recognizing this, uLangs uses a feedback-based repair loop as its central mechanism for iteratively refining these artifacts, systematically driving them towards correctness and robustness against concrete code examples. This iterative cycle stands in contrast to simpler "one-shot" generation approaches, providing a crucial layer of validation and refinement that significantly enhances the reliability of the final synthesized components. The overall process is visually represented in Figure 11.1.

The core mechanics of the loop operate similarly for both grammar and adapter synthesis, differing primarily in the specific verification methods and the nature of the feedback generated. The cycle proceeds as follows: First, the current candidate artifact (either the ANTLR grammar or the Prolog adapter) is subjected to a verification attempt using a specific code example. For grammars, this involves attempting to parse the example using the ANTLR toolchain (as detailed in Section 8.2). For adapters, it involves executing the main Prolog queries (like `holotype`) with the candidate adapter and the

linearized facts from the example, followed by validating the output against the target schema and potentially scoring its semantic accuracy using an LLM-as-a-judge (detailed in Section 9.4).

**If the verification attempt is successful** for the current example, the loop tests the same candidate artifact against the *next* available code example. However, **if the verification fails,** the process halts further testing of that candidate. The specific reason (or *reasons*) for the failure, be it an ANTLR parsing error, a Prolog execution error, a schema mismatch, or a low semantic score, is captured to form a structured *feedback signal*.

This feedback signal, the problematic code example, and the failed candidate artifact itself are then used to construct a repair prompt for the LLM. This prompt explicitly instructs the model to modify the previous candidate to address the specific issues identified in the feedback.

Upon receiving the repair prompt, the LLM generates a new, repaired candidate artifact. The loop then restarts the verification process with this new candidate, typically beginning again from the first code example (or at least the example that triggered the previous failure) to ensure the repair did not introduce regressions. This iterative cycle of verify-fail-feedback-repair-verify continues, leveraging the dialogue management strategy (Section 11.1.1) where each interaction builds upon the previous conversation history, allowing the LLM to learn from the sequence of failures and successful repairs contextually. The capabilities of modern reasoning models (Section 3.4) are particularly beneficial here, as they can often understand complex feedback more effectively and perform more substantial logical corrections in a single repair step, potentially reducing the total number of iterations required to converge on a correct solution.

Throughout this process, the system must track the state, managing the current candidate being tested and potentially referencing the last known good candidate, which is crucial for the rollback mechanism described in Section 11.2.1. The loop terminates under two conditions: either a candidate artifact successfully passes verification against all designated code examples, marking it as validated, or a predefined iteration limit is reached for a specific example, triggering the rollback-and-skip strategy for persistently problematic inputs.

This feedback-based repair loop is fundamental to uLangs' ability to produce reliable language tooling. It systematically enforces correctness against concrete examples, makes the synthesis process resilient to the inherent inaccuracies of initial LLM generations, and ultimately enables the automated production of functional and robust grammars and semantic adapters.

To further enhance the effectiveness of generated feedback, uLangs employs a strategy of overlaying error messages (if line numbers are provided) directly as inline comments within the relevant code, which is sent to the LLM model. Depending on the source of the error, these comments are inserted either into the generated grammar files or into the user-provided code snippets themselves. This approach provides the LLM with immediate "visual" context, associating each error message with the exact code construct that triggered it. Since line numbers alone offer limited utility to language models, embedding errors as inline comments significantly improves the model's ability to pinpoint and correct specific issues during the iterative repair process.

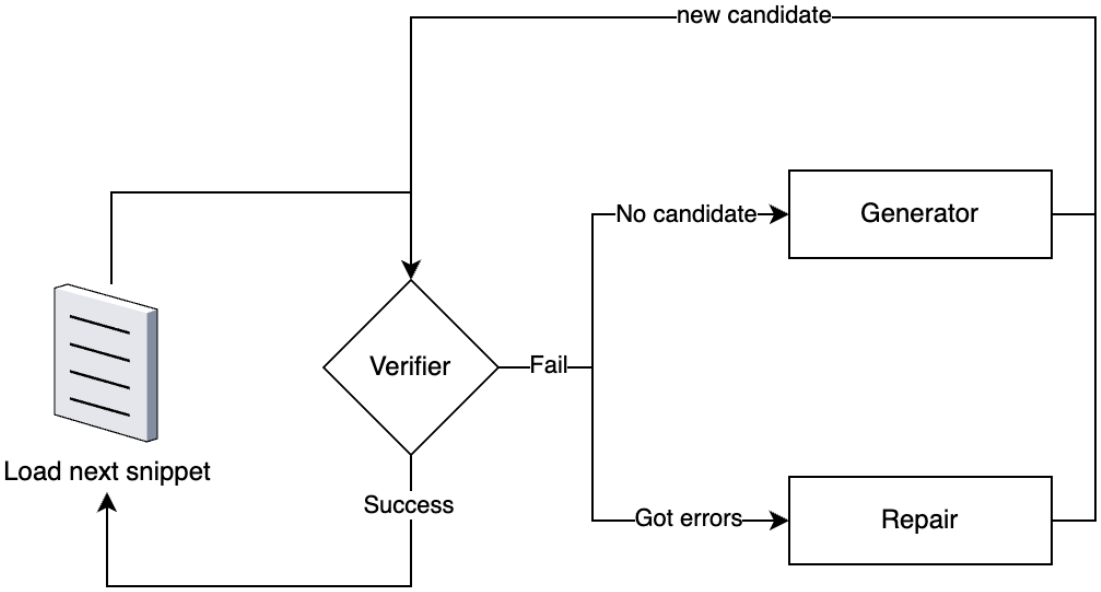



Figure 11.1: Flow the **automated generation with verifier** follows

### 11.2.1 Handling Syntax Errors in user-provided files

The uLangs synthesis process operates under the assumption that the input code examples are syntactically correct according to the target language. This allows the system to focus entirely on inferring valid grammar rules and semantic patterns from well-formed input. However, the presence of syntax errors in the provided files can disrupt this process.

Encountering a file with syntax errors typically leads to one of two outcomes during the feedback-based repair loop. Firstly, the ANTLR parser generated from the current candidate grammar may simply fail to parse the erroneous file. If this happens repeatedly, it signals an incompatibility. Secondly, and more detrimentally, the repair loop might attempt to accommodate the malformed syntax by overly generalizing the grammar rules. This attempt to reconcile valid structures with invalid ones can lead to an overly permissive grammar – one that accepts constructs that should be illegal according to the actual language specification. This is arguably the worst-case scenario, as it degrades the quality and correctness of the synthesized grammar.

To mitigate these risks, particularly the generation of overly relaxed grammars, uLangs employs a specific strategy. During synthesis, the system tracks the parsing success rate for each input file against the evolving grammar candidate. If a particular file consistently fails to parse across a configurable number of repair iterations (i.e., exceeds a failure threshold), it is flagged as potentially unreliable or syntactically incorrect. At this point, the system takes two actions:

1. **Skips** the problematic file, excluding it from further synthesis steps in the current run.

2. **Rolls back** the current grammar candidate to the last known stable checkpoint, the version that successfully parsed the largest set of other files encountered so far.

The synthesis process then continues with the next available file, using the restored grammar candidate as the starting point for the feedback-repair loop.

This rollback-and-skip mechanism prevents the synthesis from getting stuck on unparseable files and safeguards against the corruption of the grammar by problematic examples. **This resilience applies whether a file contains outright syntax errors or simply presents syntactic structures too complex to model within the allocated repair iterations.** It allows uLangs to prioritize learning from the syntactically sound and inferable examples within the input codebase.

### 11.2.2 Diff-based code edits

An early variant of the adapter repair algorithm experimented with using diff-based editing, where the language model was asked to propose modifications to the adapter or grammar by outputting a patch in the unified diff format. This approach was inspired by emerging tools such as Aider[1], which allow code to be modified through structured diffs rather than full replacements.

The motivation behind this method was to preserve more context from previous working versions of the code and to encourage minimal, focused edits. In theory, a successful patch could be applied directly to the previous version, reducing the risk of losing correct logic during the repair process.

In practice, this approach performed poorly. The language model frequently produced malformed or ambiguous diff outputs, especially when operating over longer files. Errors included invalid hunk headers and mismatched deletions or insertions. A review of the model's outputs and comments suggested that the diff format introduced confusion, making it harder for the model to reason about the intended changes. However, it is possible that better prompt engineering, more direct feedback, or further experimentation could have improved results. Similar methods appear to work well in other systems, suggesting that the failure was not inherent to the approach but may reflect limitations in the early implementation or prompt design.

No formal benchmarks were collected, but the diff-based approach led to a clear decrease in reliability and a higher rate of failed iterations during early synthesis runs. For this reason, the method was abandoned in favor of simpler, full-text repair prompts, which proved to be more robust and easier to debug.

## 11.3 Context Compression

A key challenge in automated code synthesis emerges when synthesizing intricate grammars, Prolog solutions, or when processing extensive code samples: context window saturation. This manifests through two primary limitations. First, the escalating cost of input tokens per request. This is partially mitigated through input caching mechanisms [32] employed by providers like OpenAI. These systems reduce subsequent input token costs by half when the prefix[2] is reused for later requests, within a given time period.

The more critical constraint involves reaching the chosen model's context length limits. In uLangs' current implementation, exceeding this threshold results in program termination. While legacy models like GPT-4[30] were restricted to 8,000 tokens, modern models such as *o3-mini*[31] support contexts up to 200,000 tokens, substantially increasing practical applicability.

[1] https://aider.chat/docs/unified-diffs.html

[2] Prefix is defined as previously processed input tokens from earlier requests

Our framework implements a compression pipeline through modular hooks that process the current message history. The current strategy employs pattern-based code block replacement: ANTLR grammar definitions and DSL samples (constituting $\simeq 30\%$ of the input tokens in preliminary tests) are substituted with the placeholder "*... omitted code block for brevity ...*". We maintain some temporal locality constraints to preserve critical context – the initial and final two messages remain uncompressed. While this heuristic approach doesn't eliminate context window constraints, we can assume it effectively extends the window by at least 30%.

The current compression system in place in uLangs is intentionally designed to be highly conservative in what it omits from the model's context. We ensure that the model does not immediately require any omitted information, as it will be reintroduced later when relevant to the specific sub-task (e.g., the current grammar code or code sample). This cautious approach prioritizes reliability, ensuring that critical context is always available when needed, even if it limits the overall compression potential.

This baseline implementation suggests several optimization avenues. One potential improvement could involve exploring more aggressive omission strategies, where more information is removed from the context. Proper testing would be crucial for this approach to evaluate whether such omissions negatively impact reliability or performance. If successful, this strategy could enable greater compression without compromising functionality. Additionally, an LLM could be employed to intelligently identify the most relevant information in the context and distill it into a more compact, meaningful representation. This process could involve summarizing the current state, key information, and creating structured guidance that subsequent LLM interactions can effectively use. Such *LLM-based state summarization* would enable semantic compression, moving beyond simple syntactic omission, and could potentially overcome current length limitations through iterative refinement. Both aggressive syntactic compression and semantic LLM-driven summarization represent promising directions for future research in context-aware synthesis systems.

## 11.4 File selection, splitting and diversification

The synthesis process in uLangs depends on a set of code snippets drawn from an example codebase written in the target language. These snippets are the basis for grammar inference, adapter generation, and query validation. How these files are selected, prepared, and organized has a direct impact on the stability and generality of the resulting language description.

### 11.4.1 Selection and sorting

File selection is performed recursively over a user-specified directory, filtered by file extension. All matching files are treated as candidate inputs. The selected files are then converted into snippets and sorted using the system's complexity heuristic (see Section 11.4.4). In general, shorter or structurally simpler files are prioritized first. This ordering helps stabilize the early iterations of the synthesis loop, where partial grammars or adapters are more likely to fail on complex input. This approach resembles the concept of *curriculum learning* from machine learning, where models are initially trained on simpler examples before gradually progressing to more complex ones. Ideally, the snippet set should span a range of difficulty, from minimal examples to full-featured cases, with a preference for including small and diverse inputs early in the process.

### 11.4.2 Splitting

Although uLangs operates on small code snippets, it does not currently include an automated mechanism for splitting larger files into meaningful fragments. At present, this step is manual: users must prepare individual snippet files themselves. Each file should ideally contain a small, self-contained example of the language's syntax and semantics. These examples form the basis for both grammar inference and adapter synthesis.

Automated snippet extraction is feasible, but not implemented in uLangs. A promising direction is to use a language model to suggest semantically meaningful split points based on examples of existing well-formed snippets. This approach could leverage in-context learning (Section 3), given a few good examples, the model could attempt to divide a larger file into similarly scoped fragments. To avoid discarding relevant context or introducing ambiguity, both the original file and its fragments could be retained in the input set. This redundancy would help mitigate errors in the split logic.

In addition, automatic splitting could benefit from lightweight similarity filtering. By identifying syntactically similar fragments (based on token sequences, string distance metrics, etc.), the system could help cluster related examples. This would allow a language model to better apply in-context learning by presenting similar, well-formed examples alongside the target code when attempting to generate a split. Using known-good snippets as prompts for splitting new files increases the chance of producing valid, semantically meaningful fragments.

Separately, **similarity filtering could also play a role in the synthesis process itself.** Instead of sorting snippets solely by complexity, the system could prioritize them based on similarity to previously successful examples. Once a snippet parses cleanly or produces a valid semantic result, structurally similar snippets could be scheduled earlier in the next synthesis iteration. This local feedback strategy would encourage generalization across clusters of related examples before moving on to more distinct or irregular cases, potentially improving convergence and reducing redundant repair attempts.

Heuristic approaches to splitting are also possible. Indentation levels, statement delimiters, or syntax tree structure could be used to identify candidate regions. However, such rule-based methods will likely fail on irregular or unconventional DSLs and require language-specific tuning to avoid problematic code fragmentation.

In all cases, snippet quality affects synthesis performance. Poorly scoped or overly large snippets can introduce noise and make adapter repair more difficult. Conversely, overly fine-grained fragments may lack enough structure to trigger useful generalizations. While uLangs assumes well-prepared inputs for now, future versions could incorporate an optional preprocessing stage to automate splitting while preserving diversity and structural integrity.

### 11.4.3 Diversification

The current system does not implement any diversification mechanism. As a result, redundant or near-duplicate snippets may be included in the synthesis set. The absence of diversification can significantly impact the efficiency and effectiveness of the synthesis process. Specifically, including many similar or redundant snippets can quickly bloat the context window of the LLM, increasing computational costs and slowing down inference. Additionally, presenting multiple near-identical examples in sequence may cause the LLM to become overly specialized or stuck in a local minimum, making

it harder for the iterative repair loop to guide the model toward a more general and robust solution. Finally, verifying numerous similar snippets is inherently inefficient, as it consumes computational resources without providing proportional gains in coverage or generalization.

A diversification step would help reduce this redundancy by identifying and downweighting similar snippets. This could be achieved through simple text similarity measures, such as token overlap, line-wise edit distance, compression-based, or more advanced embedding-based (semantic similarity) methods.

In some cases, full deduplication may be undesirable. Even small variations in similar examples can surface edge cases or inconsistencies in the grammar or adapter. As such, any future diversification algorithm should strike a balance between eliminating true duplicates and preserving structural variety. In the absence of such a filter, uLangs currently errs on the side of completeness, testing all provided snippets during synthesis.

Together, file selection, splitting, and diversification shape the example distribution seen by the synthesis engine. While only selection and ordering are currently supported, the architecture is designed to accommodate further improvements in preprocessing and example curation.

### 11.4.4 Complexity heuristics

During the synthesis process, the order in which code snippets are evaluated affects the stability and reliability of the generation loop. uLangs uses lightweight heuristics to estimate the relative complexity of each snippet and sorts them accordingly before synthesis begins. Simpler examples are tested first, and more complex ones are deferred until the system has had a chance to generate a working grammar and adapter on easier input.

This heuristic ordering improves convergence. In the early iterations of the generate–verify–repair loop, the system is more likely to produce incomplete or brittle solutions. By testing those against shorter or structurally simpler inputs, it reduces the chance of cascading errors and creates a gentler gradient for iterative improvement.

The default heuristic used in uLangs is based on compression. Each snippet is compressed using GZIP, and the size of the compressed output serves as a proxy for complexity. This technique captures a combination of structural variety, token diversity, and raw length without requiring language-specific knowledge. Other heuristics are supported, including line count and token count, but GZIP has proven effective as a general-purpose default.

The snippet sorting algorithm is implemented as a preprocessing step before grammar or adapter synthesis. Once the relevant files are identified in the input directory, they are converted into snippet objects, scored using the chosen complexity heuristic, and then sorted in ascending order. The inference loop then processes these in sequence.

# Chapter 12

# Challenges and Trade-offs

## 12.1 Choosing a parser generator

Selecting an appropriate parser generator was a critical design decision for uLangs, directly influencing the effectiveness of grammar synthesis, later semantic integration, and the iterative feedback-based repair loop. Early on during planning, we identified several key requirements specific to uLangs' architecture that the parser generator needed to satisfy:

- **Concise grammar definitions:** To facilitate reliable grammar synthesis using LLMs, the grammar specification language needed to be compact and straightforward.
- **Clear parse tree representation:** The generated parse trees had to map cleanly into Prolog facts, enabling seamless integration with the semantic reasoning layer.
- **High-quality error reporting:** Detailed and actionable parsing errors were essential, as they serve as the primary feedback signal driving the iterative grammar repair process.
- **Ease of integration:** The parser generator had to support TypeScript as a target language or provide straightforward bindings.

ANTLR was chosen because it aligns closely with these requirements and provides a strong foundation for uLangs' modular and iterative design. ANTLR grammars are concise and self-contained, typically defined in just two concise files (lexer and parser). This simplicity is ideal for LLM-driven synthesis, as it minimizes the risk of errors or inconsistencies in generated grammars. ANTLR also produces explicit and consistent parse trees, which can be easily translated into Prolog facts, ensuring smooth integration with the semantic reasoning layer. Its detailed error reporting provides actionable feedback during the repair loop, allowing the system to refine grammar candidates effectively. Additionally, ANTLR's maturity (making it known for LLMs), extensive community support, and broad language targeting (including TypeScript), further reinforced its suitability. Although grammars are mainly constructed with EBNF-style syntax, they also support embedded actions (see Section 4.2) which offer flexibility for more advanced extensions, making it adaptable to evolving synthesis techniques.

Tree-sitter was also considered as an alternative parser generator. Its incremental parsing and robust error recovery mechanisms initially appeared beneficial, especially given uLangs' goal of supporting IDE tooling. Additionally, Tree-sitter grammars

are defined in JavaScript, a language that LLMs handle well, potentially simplifying grammar synthesis. However, Tree-sitter generates parser code in C, requiring additional bindings to integrate with TypeScript, which would have complicated the toolchain. While incremental parsing was appealing, ANTLR's error recovery proved sufficient for uLangs' current needs, and its straightforward compatibility with Prolog was ultimately more important. Given these trade-offs, ANTLR was the more practical choice for uLangs' architecture and objectives.

### 12.1.1 Using ANTLR as checking and runtime

uLangs relies on ANTLR in two complementary ways: first, as a grammar-checking mechanism during synthesis, and second, as the parser runtime within the generated language server. These two uses share the same grammar specification but differ in their implementation details and integration points.

For grammar checking, uLangs employs a custom Java program adapted from the ANTLR Lab repository[1], designed specifically to verify generated grammars. This checker compiles candidate grammars and attempts to parse the provided code snippets, producing detailed parsing error messages and location metadata if parsing fails. These errors become the primary feedback signal guiding the iterative grammar repair process. By clearly identifying invalid grammar rules or ambiguous constructs, the checker helps the system efficiently pinpoint and resolve synthesis issues.

On the runtime side, uLangs uses ANTLR's generated TypeScript parser classes directly within uLangs' TypeScript codebase. The emitted parser code is executed at runtime to parse user-provided files into concrete syntax trees (CSTs). These parse trees are subsequently linearized into Prolog facts, forming the basis for semantic reasoning and query execution. This runtime simplifies integration and removes the need for complex language bindings (like in tree-sitter). Although the ANTLR runtime parser lacks the custom error-reporting sophistication of the Java-based checker, it offers some error handling to gracefully handle unexpected or partially valid user input at runtime.

### 12.1.2 Issues with indentation-based languages

Languages like Python use **significant whitespace**, meaning indentation defines code blocks, in contrast to languages using delimiters like braces or keywords. ANTLR does not have a native way to deal with the *INDENT* and *DEDENT* tokens we need to handle. At the time of writing (using ANTLR4) the only solution is to use ANTLR Actions (see Section 4.2, which allows you to embed code (like Java or Javascript) directly into the ANTLR grammar. This increases the complexity and error surface substantially as in that case, the LLM has to write intricate lexing and parsing code in the set target language without being constrained by the ANTLR grammar language. uLangs can still successfully create a working solution grammar without using actions but the scopes will not be properly represented by the generated tree after parsing.

A more robust solution here could be to provide some ready-to-use indentation logic in the same it is implemented by the *Python3* grammars created by the ANTLR team. As we only need to support one target language, this wouldn't be too difficult. The language as used by the uLangs query components is *Typescript* so that became also the set target language.

[1] ANTLR Lab is an online playground to experiment with ANTLR grammars. Repository can be found here: https://github.com/antlr/antlr4-lab

## 12.2 Choosing SWI-Prolog

Following the syntactic analysis performed by the ANTLR-generated parser, uLangs requires a robust and expressive mechanism to conduct semantic analysis. This involves interpreting the structural information captured in the parse tree to understand deeper language constructs such as symbol declarations, references, scopes, and potentially type relationships. For this critical task, the Prolog programming language was selected as the foundation for uLangs' semantic reasoning engine.

### 12.2.1 Rationale for Prolog in Semantic Analysis

The decision to use Prolog for semantic analysis stems directly from its inherent strengths as a logic programming language, as detailed in Chapter 5. Its declarative nature allows semantic rules, which are synthesized by the LLM in uLangs, to be expressed as logical relationships rather than imperative procedures. **This focus on what semantic constructs exist, rather than how to algorithmically find them, provides a more natural and potentially more manageable target for LLM generation.**

Prolog's core mechanism of unification offers powerful pattern-matching capabilities. Given that the linearized parse tree (derived from ANTLR's output) is represented as a collection of Prolog facts, unification is exceptionally well-suited for identifying specific syntactic patterns within these facts that correspond to meaningful semantic elements. For instance, rules can be defined to match the structure of a function declaration or a variable assignment within the fact base. Coupled with its built-in backtracking and inference engine, Prolog can then efficiently execute complex semantic queries, such as resolving symbol definitions or finding all references, based on the synthesized adapter rules. This ability to declaratively define rules and have the system infer consequences is a key advantage when using Prolog

### 12.2.2 Selecting SWI-Prolog as the Implementation

Once Prolog was chosen as the language paradigm for semantic reasoning, a specific implementation was required. **SWI-Prolog** was selected for uLangs due to its maturity, practical features directly relevant to the project, and its open-source nature [49]. As a widely used and actively maintained system, SWI-Prolog provides a stable and reliable foundation for semantic analysis.

Before settling on full Prolog, *Datalog*, a subset of Prolog, was also briefly considered. Datalog offers several attractive properties, including a simpler syntax, guaranteed termination for queries, and potentially more efficient evaluation for certain classes of recursive queries often found in static analysis. Its focus on relational data and deductive database capabilities seemed promising for representing and querying code structures. However, for the broader goals of uLangs, particularly the need to synthesize potentially complex semantic transformation rules and to handle tasks like JSON manipulation for output, the greater expressiveness and more extensive general-purpose libraries of full Prolog, as implemented in SWI-Prolog, were deemed more practical. While Datalog excels at certain analytical tasks, the flexibility to define more intricate predicates and interact with external data formats made Prolog a more versatile choice for the diverse requirements of the semantic adapter and the overall system.

A critical factor in the decision for SWI-Prolog was its **excellent support for JSON processing**. This capability is indispensable for uLangs, as the main semantic queries (such as `holotype` and `definitions`) are designed to output their results in structured

JSON format. This JSON output is consumed by downstream components, including the Universal LSP Server, and is also used by the LLM-as-a-judge mechanism during semantic adapter synthesis for exchanging scores and feedback. SWI-Prolog's built-in libraries for parsing and generating JSON greatly simplify these interactions.

Furthermore, SWI-Prolog's **ease of integration** into the uLangs workflow was a key consideration. uLangs, primarily developed in TypeScript, interacts with the Prolog engine by invoking it as a subprocess via its command-line interface. SWI-Prolog is well-suited for this model, efficiently handling the loading of linearized parse tree facts and the synthesized adapter, executing the queries, and returning JSON output via standard streams. This straightforward integration method allowed for a clean separation between the TypeScript-based control logic of uLangs and the Prolog-based semantic reasoning core. While SWI-Prolog offers a wide array of advanced features, for uLangs, the primary benefits stem from its reliable core Prolog engine, good-enough performance, robust JSON handling, and ease of use as a command-line tool. These characteristics, combined with its maturity and open-source availability, made SWI-Prolog the most practical and effective choice for implementing the semantic analysis component of the uLangs toolkit.

## 12.3 Mitigating issues with example-driven artifact synthesis

Synthesizing grammars and semantic adapters directly from code examples using LLMs introduces challenges analogous to common pitfalls in machine learning, particularly *overfitting* and getting stuck in *local minima*. Overfitting, in this context, means the synthesized artifacts become overly specialized to the specific examples provided, failing to generalize correctly to new, unseen code. Similarly, local minima refer to situations where the synthesis process settles into a suboptimal solution space, making it difficult to incrementally improve or correct the generated artifacts through iterative feedback alone.

In practice, we've observed that modern LLMs are surprisingly robust against these issues, especially when given enough opportunities to reconsider and rewrite their solutions. Unlike traditional optimization algorithms, LLMs can often escape local minima by significantly revising their approach when prompted with useful feedback signals. However, the risk of overfitting remains relevant, particularly when the number of provided code snippets is limited. We generally assume users have access to reasonably sized codebases or collections of examples, which naturally reduces the likelihood of severe overfitting.

To further mitigate these risks, uLangs employs several complementary strategies throughout its synthesis pipeline. As briefly mentioned in Section 8.1, the initial grammar synthesis step explicitly encourages the LLM to produce general solutions. By providing all available examples simultaneously during the initial generation, the model is prompted to find common syntactic patterns that generalize across the entire set. This holistic initial guess contrasts with the subsequent repair phase, where the model addresses parsing failures on an example-by-example basis. Additionally, the ANTLR checker (Section 8.2) provides a strong verification signal, ensuring that grammars at least compile and produce parse trees. While passing this check doesn't guarantee perfect generalization, it significantly reduces the risk of overly narrow or brittle grammars, and in practice it has shown to be quite effective.

**Semantic adapter synthesis presents a more complex challenge.** Unlike ANTLR grammars, which LLMs typically understand well from pre-training, generating Prolog adapters to map arbitrary syntax encoded as parse trees to structured

semantically useful outputs is a novel and less familiar task. Consequently, we rely on more robust guardrails and feedback mechanisms. Basic checks, such as verifying valid Prolog syntax, correct JSON output, and schema adherence, are necessary but insufficient on their own. Formally verifying the semantic correctness of the generated adapters is inherently challenging, as there is no "ground truth" available for arbitrary user-provided languages.

To address this verification gap, uLangs incorporates an LLM-as-a-judge scoring mechanism within the semantic feedback loop (Section 9.4). After initial structural checks, this scorer evaluates the semantic plausibility of the adapter's output relative to the original code snippet. It assigns a qualitative score and provides explicit textual feedback, highlighting specific issues such as missing symbols, incorrect AST structures, or irrelevant references. This targeted feedback helps guide the LLM toward generating adapters that not only conform structurally but also make semantic sense, significantly reducing the risk of semantic overfitting or nonsensical outputs.

## 12.4 Advanced search algorithms

### 12.4.1 Trunk-Based Parallel Sampling

In the early stages of this research, we explored a grammar inference approach we termed *trunk-based parallel sampling*. This method aimed to leverage the generative capabilities of LLMs to automatically synthesize ANTLR lexer and parser grammars from provided code examples. The primary motivation behind *this* approach was to efficiently explore multiple candidate grammars in parallel, systematically refining them based on parsing feedback, and progressively converging toward a robust grammar capable of parsing increasingly complex code examples.

The trunk-based parallel sampling approach is structured around a linear progression of intermediate solutions, metaphorically referred to as the *trunk*[2]. Each intermediate solution represents a lexer-parser grammar pair that successfully parses all code examples encountered up to that point. The inference process begins with the simplest available code example, establishing an initial intermediate solution. Subsequently, the process iteratively advances along the trunk, incrementally introducing more complex code examples at each step.

At each incremental step, the inference process proceeds as follows:

1. **Candidate Generation:** Starting from the current intermediate solution (or an initial candidate for the first step), the LLM generates multiple candidate grammars in parallel. Typically, tens to hundreds of candidates are produced simultaneously, each representing a distinct lexer-parser grammar pair. To encourage diversity among candidates, the LLM is configured with a relatively high sampling temperature.

2. **Candidate Evaluation:** Each candidate grammar is independently tested against the current code example. The evaluation criterion is straightforward: a candidate is considered successful if it correctly parses the provided code without errors.

3. **Parallel Refinement:** If all candidates fail to parse the current code example, each candidate undergoes parallel refinement. Specifically, parsing errors produced by the ANTLR toolchain are collected individually for each candidate. These errors

[2] *Trunk* term borrowed from the version control management practice of trunk-based development

are then provided as feedback to the LLM within each candidate's own isolated message history, prompting the generation of refined grammars. This parallel refinement step is repeated iteratively until at least one candidate successfully parses the current code example or a predefined iteration limit is reached.

4. **Intermediate Solution Selection:** When multiple candidate grammars successfully parse the current code example, a heuristic based on grammar complexity is employed to select the optimal intermediate solution. Complexity is quantitatively measured using the compressed size of the grammar files, specifically the number of bytes after GZIP compression. The candidate grammar with the lowest complexity score is chosen as the intermediate solution for the current step.

5. **Progression to Next Step:** The selected intermediate solution becomes the starting point for the subsequent step, where the next, more complex code example is introduced. The process then repeats from step 1.

This iterative, trunk-based progression ensures that each intermediate solution is capable of parsing all previously encountered code examples, thereby incrementally building a robust grammar capable of handling increasingly complex syntax. The parallel sampling and refinement strategy significantly increases the likelihood of rapidly identifying viable grammars at each step, while the complexity heuristic ensures that the resulting grammars remain manageable and maintainable.

From a technical perspective, parallelism was implemented using asynchronous programming constructs provided by Node.js promises. Each candidate grammar maintained its own isolated LLM message history, ensuring that refinement feedback remained contextually relevant and specific to each candidate's parsing errors.

Despite its theoretical advantages and initial successes, the trunk-based parallel sampling approach exhibited significant practical limitations. Most notably, it required a considerable number of parallel samples and extensive code examples to reliably converge on a functional grammar. Furthermore, the method struggled considerably when faced with more complex grammars, often failing to efficiently navigate the increasingly large search space. These challenges highlighted the need for a more sophisticated generation and search algorithm, motivating the exploration of alternative approaches such as Monte Carlo Tree Search (MCTS), discussed in the following chapter. However, the eventual replacement of trunk-based parallel sampling was primarily driven by the emergence of a new class of large language models, reasoning models, which demonstrated remarkable few-shot learning capabilities. These reasoning models could efficiently synthesize grammars from fewer examples and required only minimal iterative feedback, effectively eliminating the need for extensive parallel sampling. Interestingly, the core iterative refinement concept from trunk-based parallel sampling proved valuable and was retained in the new approach, albeit simplified into a single, linear refinement timeline without parallel candidate generation.

### 12.4.2 Monte Carlo Tree Search

Given the limitations encountered with the trunk-based parallel sampling approach, particularly its inefficiency in handling complex grammars and the substantial computational overhead, we explored more advanced search algorithms to better navigate the grammar inference search space. One promising candidate was Monte Carlo Tree Search, a probabilistic search algorithm widely used in domains characterized

by large, complex search spaces, such as game playing and combinatorial optimization problems.

Monte Carlo Tree Search is a probabilistic search algorithm designed to efficiently explore large and complex search spaces (such as grammar synthesis), by iteratively building a search tree through random sampling. At each iteration, MCTS selects promising candidates, expands the tree by generating new solutions, evaluates them through simulations, and updates the tree to guide future searches toward better regions of the tree. [9].

Inspired by recent work by Brandfonbrener et al. [7] and Robert Shin and Arun Patro [38] applying MCTS to language-model-driven generation tasks, we began implementing an MCTS-based grammar synthesizing pipeline. The envisioned approach involved integrating a verifier and scorer into the MCTS framework, enabling the algorithm to systematically explore and evaluate candidate grammars generated by the LLM. The verifier would assess parsing correctness, while the scorer would quantify grammar complexity and quality, providing meaningful feedback to guide the search.

The primary motivation behind adopting MCTS was its potential to more efficiently allocate computational resources, such as LLM requests and token usage, by intelligently focusing exploration on the most promising candidate grammars. Additionally, MCTS's structured search approach could potentially allow the use of smaller, simpler language models or even enable fine-tuning specialized models tailored specifically to grammar inference tasks.

However, the implementation of the MCTS-based approach was ultimately halted due to significant advancements in language modeling technology. Specifically, the emergence of reasoning models (see Section 3.4), such as OpenAI o1 or DeepSeek R1, demonstrated remarkable few-shot learning capabilities. These models could efficiently synthesize accurate grammars from minimal examples, requiring only limited iterative feedback. This development dramatically simplified the grammar inference pipeline, rendering the complexity and computational overhead of MCTS unnecessary for our practical purposes.

Consequently, while the MCTS approach was not pursued to completion, the exploration provided valuable insights into structured search methodologies and informed our understanding of the trade-offs involved in grammar inference. Future research may revisit MCTS or similar structured search algorithms, particularly in scenarios where reasoning models are unavailable, computational resources are constrained, or fine-tuned smaller models are preferred.

## 12.5 Model choice

There are many considerations when choosing between different LLM models, because of this uLangs was designed to support **any model** that can be provided through the OpenAI-compatible API and supports Structured Output (see 3.5). The best-performing models seem to be reasoning models (see 3.4). The official choice for uLangs is *o3-mini* as it performs as well or better than o1 but is faster and 13 times cheaper on OpenAI's current pricing [31]. The excellent performance of reasoning models enabled the uLangs generation pipeline to be simplified significantly compared to the planned search algorithms that were originally considered.

However, synthesizing the *semantic adapter* is a notably more challenging task for LLMs. In practice, we have observed significantly better results and faster convergence

when using larger reasoning models, such as OpenAI's full-scale *o3* model[3]. We theorize that this difference arises from several factors:

- The task itself, generating Prolog code to map arbitrary syntactic structures (represented as facts) to a specific semantic schema, is novel and highly specialized, likely rare or absent in the models' original training data.
- Prolog itself might be underrepresented in the vast datasets used for pre-training compared to more mainstream programming languages, limiting the model's inherent fluency.
- The feedback signals provided during semantic synthesis (like schema mismatches or LLM-as-a-judge scores) are inherently more abstract and less directly actionable compared to the concrete syntax errors reported during grammar verification.

It's important to acknowledge that the current prompting strategies and system parameters may be implicitly optimized for the models predominantly used during development (primarily OpenAI's *o3* and *o3-mini*). We observed, for instance, that the LLM-as-a-judge scorer exhibits notable sensitivity to the underlying model. When experimenting with OpenAI's newer *o4-mini* model as the scorer instead of the *o3-mini* model, it tended to be significantly "stricter," assigning consistently lower scores for similar adapter outputs. This sensitivity primarily becomes an issue due to the current hardcoded threshold for the minimum acceptable score. Adapting uLangs to leverage different models optimally might therefore require recalibrating such thresholds or even fine-tuning the prompts[4] and feedback interpretation logic.

### 12.5.1 Token use and cost

Using LLMs in uLangs introduces practical considerations related to token consumption and associated monetary costs. LLM usage is typically metered in *tokens*, which include both input prompts and generated outputs. Given uLangs' iterative generate-verify-repair loop (Section 11.2), token usage can accumulate significantly, as each iteration involves sending prompts (including feedback and conversation history) and receiving generated artifacts. Several factors influence token consumption in uLangs. The choice of model (Section 12.5) directly impacts cost, as larger models like OpenAI's *o3* are more expensive per token compared to smaller models like *o3-mini*. Additionally, the complexity of the target language or task can increase token usage, as more intricate examples or detailed feedback require longer prompts and potentially more repair iterations. The length of the conversation history also contributes to the token count, emphasizing the importance of context compression strategies (Section 11.3) to manage prompt size.

Concrete numbers from the Lua case study (in Chapter 13, Table 13.1) illustrate these considerations clearly. Grammar synthesis using the smaller *o3-mini* model, in addition to requiring relatively few tokens and API calls, also has much lower per-token pricing [31], resulting in modest costs. In contrast, semantic adapter synthesis, which employed the larger *o3* model due to task complexity, incurred significantly higher token usage and associated costs.

Overall, token usage and cost represent important practical trade-offs in uLangs. Balancing the use of more capable but expensive models against operational efficiency

[3] https://openai.com/index/introducing-o3-and-o4-mini/

[4] Our prompts can be found in appendix A

is crucial. Future work, such as fine-tuning smaller specialized models (discussed in Chapter 14), could further optimize token consumption and reduce costs.

# Part III

# Evaluation and Conclusion

# Chapter 13

# Evaluation

The evaluation focuses specifically on two common LSP methods: Go to Definition (textDocument/definition) and Find References (textDocument/references). These methods were chosen because they directly correspond to the semantic information captured by the holotype query, which was central to the semantic adapter synthesis process described earlier. Additionally, these methods have practical importance in software development workflows and rely heavily on accurate syntactic and semantic analysis, making them good candidates for evaluating the effectiveness of the synthesized *LDS*.

## 13.1 Metrics

### 13.1.1 *Go to Definition* Accuracy

The Go to Definition method is expected to return exactly one location: the precise definition of the symbol at the requested position. Therefore, we measure the correctness of this method using a simple binary accuracy metric:

$$\text{Accuracy} = \frac{\text{Number of correctly identified definitions}}{\text{Total number of definition requests}}$$

A definition request is considered correct if and only if the location returned by the synthesized LDS exactly matches the golden standard location (file path, line number, and column range).

### 13.1.2 *Find References* Metrics

The Find References method returns a set of locations where a given symbol is referenced throughout the codebase. Unlike definitions, multiple locations may be returned, and the correctness of the result set can be partially correct. Thus, we use standard information retrieval metrics: *Precision*, *Recall*, and *F1-score* to evaluate performance.

**Precision** Precision measures the proportion of returned references that are correct (i.e., actually references according to the golden standard):

$$\text{Precision} = \frac{\text{Number of correctly identified references}}{\text{Total number of references identified by uLangs}}$$

A high precision indicates that the synthesized LDS rarely returns incorrect or irrelevant references.

**Recall** Recall measures the proportion of actual references (according to the golden standard) that were successfully identified by the synthesized LDS:

$$\text{Recall} = \frac{\text{Number of correctly identified references}}{\text{Total number of actual references in the golden standard}}$$

A high recall indicates that the synthesized LDS rarely misses actual references.

**F1-score** The F1-score is the harmonic mean of precision and recall, providing a balanced measure of performance:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-score is particularly useful when precision and recall are both important, as it penalizes extreme imbalances between the two.

**Performance Evaluation Across Multiple Queries** To provide a comprehensive view of the *Find References* performance across multiple test cases (i.e., queries for different symbols), we employ two common averaging strategies for Precision, Recall, and F1-score:

- **Micro-averaging:** This method computes the metrics globally by summing the total number of True Positives (TP), False Positives (FP), and False Negatives (FN) across all items from all test cases before calculating Precision, Recall, and F1-score. In the context of "Find References", micro-averaging gives more weight to queries that involve a larger number of individual reference items. A high micro-averaged score indicates strong overall performance when considering the total volume of correctly identified (or missed/incorrectly identified) references across all queries.
- **Macro-averaging:** This method calculates Precision, Recall, and F1-score for each test case (i.e., for each symbol query) independently and then computes the unweighted average of these per-test-case scores. Macro-averaging gives equal weight to every query, regardless of how many reference items are associated with each symbol. A high macro-averaged score suggests that the system performs consistently well across different types of queries, irrespective of the volume of references per query.

Both micro-averaged and macro-averaged results will be presented in our evaluation to offer a nuanced understanding of the synthesized LDS's performance. Micro-averages will reflect its bulk performance on all reference instances combined, while macro-averages will highlight its consistency across different symbol queries.

### 13.1.3 Golden Standard

To objectively evaluate the uLangs outputs, we require a reliable "golden standard" against which to compare the results. For this evaluation, we use an established and widely adopted Lua language server implementation, specifically the lua-language-server. This server is mature, widely used in industry, and considered reliable for Lua development.

Each evaluation request (definition or references) is executed against the lua-language-server, and the resulting locations are recorded as the golden standard. This approach ensures objectivity and reproducibility in our evaluation.

## 13.2 Evaluation Procedure

The evaluation procedure consists of the following steps:

1. **Gathering Examples:** Gather a diverse set of examples in our target language, these will be used for synthesis and evaluation.

2. **Synthesis Phase:** Generate the LDS (ANTLR grammar and Prolog semantic adapter) for Lua using a set of Lua code examples. The evaluation file is explicitly excluded from this synthesis set.

3. **Test Case Design:** Select a complex Lua file (unseen during synthesis) and design realistic LSP requests (definition and references) targeting various symbols and language constructs.

4. **Golden Standard Execution:** Execute each designed LSP request against the `lua-language-server`[1] to get the golden standard results.

5. **uLangs Execution:** Execute the same LSP requests using the synthesized LDS via the uLangs runtime

6. **Naive Client Execution:** Execute the same LSP requests using a naive baseline LSP client implementation that simply uses string matching.

7. **Metrics Calculation:** Compare the uLangs results against the golden standard and the naive baseline, then calculate the metrics defined above.

## 13.3 Qualitative and Results

### 13.3.1 Case Study: Lua

#### Choosing a programming language

Lua was selected as the primary case study language for this evaluation due to several key characteristics that make it a suitable candidate for testing the uLangs synthesis pipeline. Firstly, Lua strikes a practical balance: its core syntax is relatively straightforward, making the initial grammar synthesis task tractable, yet it possesses a rich set of semantic features, such as versatile tables used for object-oriented patterns, lexical scoping, and first-class functions, which provide a meaningful challenge for semantic adapter synthesis. Secondly, the availability of the mature `lua-language-server` offers a crucial "golden standard" against which the performance of the uLangs. Finally, Lua's use in diverse applications ensures that a reasonable corpus of code examples can be sourced for the synthesis process. These factors combined make Lua a representative language for demonstrating and evaluating the capabilities of the uLangs automated synthesis approach.

[1] Repository found at https://github.com/LuaLS/lua-language-server

### Synthesizing the artifacts

As outlined in the evaluation methodology (Section 13.2), the first step involved synthesizing the LDS for Lua. To achieve this, a corpus of representative Lua code examples was required. We sourced these examples from the code repository accompanying the Lua Quick Start Guide [2], originally intended for teaching purposes. This repository was selected because it provides concise, self-contained examples covering a broad range of Lua language features, making it well-suited for synthesizing an LDS.

All *.lua* files were extracted from this repository. A manual curation process followed, removing snippets known to contain deliberate syntax errors or designed to be incomplete examples. This resulted in a set of 99 Lua code snippets used for the synthesis phase. For reproducibility, this dataset is available within the uLangs codebase under the `test-dsl/lua-med` directory.

The synthesis process began with the generation of the ANTLR grammar rules. For this task, the OpenAI o3-mini model was employed. The grammar synthesis converged efficiently, requiring relatively few API calls and generating grammar files capable of successfully parsing all 99 snippets (100% pass rate), as detailed in Table 13.1. It is worth noting, however, that the LLM's performance in generating artifacts for Lua might be influenced by its pre-existing knowledge of the language, given Lua's relative popularity and the potential presence of existing Lua ANTLR grammars within the model's training data. This consideration is particularly relevant for the synthesis of the ANTLR grammar.

Subsequently, the Prolog semantic adapter was synthesized. Given the increased complexity of mapping syntactic structures to semantic representations (as discussed in Section 12.5), the larger and more capable OpenAI o3 model was utilized for this stage. To account for potential generation variability inherent in LLMs, the adapter synthesis process was executed three times independently. The adapter yielding the highest pass rate against the synthesis snippets was selected for further evaluation.

The results from the three runs were largely consistent, with one outlier achieving an 85/99 pass rate. The best-performing run produced an adapter that successfully processed 92 out of the 99 snippets (93%) within the configured limit of **three repair iterations** per snippet. While not achieving complete coverage on the synthesis set under these constraints, this adapter represented the most effective semantic representation generated and was thus carried forward for the functional evaluation presented in the next section.[3]

Table 13.1 summarizes the key statistics for both the grammar and the best-performing semantic adapter synthesis processes, including the models used, token consumption, API call counts, and usage costs[4].

Table 13.1: Synthesis process statistics

| Artifact | Pass rate | Attempts | Main model | Scorer model | Tokens usage (main) | Tokens usage (scorer) | API calls | Cost ($) |
|---|---|---|---|---|---|---|---|---|
| **ANTLR grammar** | 99/99 (100%) | 1 | o3-mini | — | 433 671 | — | 14 | 0.39 USD |
| **Semantic adapter** | 92/99 (93%) | 3 | o3 | o3-mini | 588 449 | 2 514 251 | 177 | 6.85 USD |

[2]Repository found at https://github.com/PacktPublishing/Lua-Quick-Start-Guide

[3]Since LLM-based synthesis is inherently non-deterministic, you can find the specific generated artifacts for this experiment in `test-dsl/lua-med/solution` within the uLangs repository.

[4]Costs were found in the OpenAI dashboard for the uLangs project and compared to expected costs

### Finding a golden standard sample

To evaluate the functional capabilities of the synthesized Lua LDS, a "golden standard" code sample was required. This sample needed to be representative of typical Lua code, containing a variety of common language constructs, and be of a manageable size for manual analysis and interaction definition. The selection process mirrored the approach used for sourcing the synthesis data: a GitHub search was conducted using the query "tutorial language:Lua", with results sorted by "Most stars". The first repository that appeared to be a substantial Lua project, rather than just a collection of snippets, was selected. This led to the "BYTEPATH" repository by user *a327ex*[5], described as "A replayable arcade shooter with a focus on build theorycrafting."

Within this repository, the first `.lua` file encountered, `GameObject.lua`, was chosen as the golden standard sample (See Appendix C.1). This file met the desired criteria: it contained several common Lua constructs, including object-like tables, functions, loops, and conditional statements. Furthermore, its length of 62 lines of code fell within the target range of 50 to 100 lines, making it suitable for detailed manual inspection.

Once this golden standard file was selected, the next step involved defining a set of reasonable user interactions that a developer might perform when working with this code. These interactions were designed to map directly to common LSP methods, specifically `textDocument/definition` (go to definition/declaration) and `textDocument/references` (find references). A total of 20 distinct interactions were manually defined: 10 for testing definition lookups and 10 for testing reference finding. These interactions involved selecting various symbols (variables, function names, table keys) within `GameObject.lua` and identifying their expected definitions or all their references within that file. It is important to note that while an effort was made to create a diverse and representative set of interactions, the manual nature of this process introduces a potential for bias in their selection.

### Evaluation code

To quantitatively assess the performance of the uLangs-synthesized Lua LDS, a dedicated evaluation framework was developed, primarily in Python. This framework centers on evaluating two fundamental LSP features: `textDocument/definition` (Go to Definition) and `textDocument/references` (Find References).

The methodology employed a comparative approach, benchmarking the uLangs-generated Lua support against an established "golden standard": the mature and widely adopted `lua-language-server`. The output from this server for predefined test cases on a specific Lua file was considered the ground truth. Importantly, this Lua file was deliberately excluded from the snippets used during the synthesis of the Lua LDS, ensuring the evaluation tested the generalization capability of the synthesized artifacts on unseen code.

The uLangs system's semantic analysis of the evaluation code was pre-computed using uLangs and the synthesized LDS, with the identified declarations and references stored in a JSON file. The entire evaluation process was then orchestrated by a shell script that iterated through the manually defined test cases, invoking a comparison script for each.

[5]Repository found at https://github.com/a327ex/BYTEPATH

This comparison script obtained results from three sources:

1. The `lua-language-server` (the golden standard), queried using a dedicated Python LSP client script.
2. The uLangs system's analysis, accessed via a `mock_lsp_client.py` script. This mock client was specifically designed to read the uLangs-native output and transform it to mimic the standard LSP JSON response structure (e.g., `LocationLink` for definitions, `Location[]` for references) that the real LSP client would produce.
3. Lastly, a Naive LSP client, used as a baseline.

To provide an additional point of comparison and establish a performance baseline, a naive string-matching based LSP client (`naive_lsp_client.py`) was also implemented. This naive client identifies the token at a given location using regular expressions (`\[a-zA-Z_][a-zA-Z0-9_]*\`). For "definition" requests, the first textual occurrence of that token in the file is given as a result. For "reference" requests, it returns all locations where the token appears via string matching. Crucially, this `naive_lsp_client.py` was designed to directly produce output formatted to mimic standard LSP response structures (e.g., `LocationLink` for definitions, `Location[]` for references), similar to how the `mock_lsp_client.py` (which serves uLangs' results) operates.

The comparison script then compared these three sets of structured LSP-like responses. For `textDocument/definition`, accuracy was the primary metric. For `textDocument/references`, standard information retrieval metrics were employed: Precision, Recall, and F1-Score. These scores were both micro-averaged and macro-averaged and reported to provide a comprehensive view of performance.

It is worth mentioning that this demonstrates that the structured semantic information generated by uLangs, while in its own internal format, can be readily mapped to standard LSP responses with minimal transformation logic. This highlights how close the uLangs system is to providing an end-to-end solution, where the synthesized knowledge is nearly directly consumable by a standard LSP server implementation (as discussed in Chapter 10).

### Comparing against the golden standard

Having synthesized the Lua LDS artifacts as described previously, we now evaluate their practical effectiveness. This evaluation involves executing the predefined set of textDocument/definition and textDocument/references requests on the complex, unseen Lua evaluation file using the uLangs-generated grammar and semantic adapter.

The results produced by uLangs for each request were then compared against the golden standard results obtained from the established lua-language-server. Table 13.2 presents a detailed breakdown of this comparison, showing the outcome for each test case and the corresponding performance metrics (Accuracy for definitions; Precision, Recall, and F1-score for references).

### Comparing against a *naive* baseline

In addition to comparing the uLangs results against the `lua-language-server` (golden standard), its performance was also evaluated against a deliberately simplistic, naive baseline implementation. This baseline, implemented in `naive_lsp_client.py`, relies purely on string matching and regular expressions to provide "Go to Definition" and

"Find References" functionalities. For a `definition` request, it identifies the token (using regex) at the cursor position, then returns the first textual occurrence of that token within the file. For `references`, it similarly identifies the token and returns all locations where that exact string appears.

The purpose of this comparison was twofold:

- To establish a lower-bound performance benchmark, demonstrating the value provided by uLangs' more sophisticated semantic analysis over trivial text-based approaches.
- To quantitatively illustrate the degree to which understanding actual language syntax and semantics (as attempted by uLangs) improves upon simple string searching for code intelligence tasks.

The same metrics—Accuracy for definitions, and Precision, Recall, and F1-score (both micro and macro-averaged) for references—were calculated. The results of this comparison, detailed in Table 13.3, highlight the limitations of such a naive approach and underscore the improvements achieved by the uLangs synthesis process.

The overall accuracy using the naive `textDocument/definition` is 60% (6 out of 10).

For the `textDocument/references` requests, aggregated performance metrics using *Naive LSP Client* are as follows:

- **Micro-averaged scores**
  - **Precision:** 72.4%
  - **Recall:** 100.0%
  - **F1-Score:** 83.9%
- **Macro-averaged scores**
  - **Average Precision:** 77.3%
  - **Average Recall:** 100.0%
  - **Average F1-Score:** 84.9%

**Notes for Table 13.2 and Table 13.3:** *Target Symbol* shows the symbol and the `line:column (L:C)` of request trigger (1-based). *Golden Result*, *uLangs Result* and *Naive Result* show the number of references found for `references` method, or the correctness for definitions. P=Precision, R=Recall, F1=F1-score. N/A indicates metric is undefined.

Overall, for the `definition` requests, **uLangs achieved accuracy of 80%** (8 out of 10 requests). This is a better result than the baseline *naive* client which achieved, which was an accuracy of 60% (6 out of 10).

For the `textDocument/references` requests, the aggregated performance metrics were as follows:

- **Micro-averaged scores**
  - **Precision:** 71.9%
  - **Recall:** 97.6%
  - **F1-Score:** 82.8%

Table 13.2: LSP Performance Evaluation of **uLangs LSP Client** on Lua Code (*GameObject.lua*)

| Method | Target Symbol (Trigger Location) | Golden Result | uLangs Result | Accurate | P | R | F1 | Notes |
|---|---|---|---|---|---|---|---|---|
| definition | `GameObject` (L3:C10) | *Correct* | *Correct* | Yes | — | — | — | Correct class definition. |
| definition | `area` (L7:C17) | *Correct* | *Correct* | Yes | — | — | — | Correct parameter definition. |
| definition | `GameObject` (L19:C10) | *Correct* | *Correct* | Yes | — | — | — | Correct method class context. |
| definition | `self` (L20:C8) | *Correct* | *Incorrect* | No | — | — | — | uLangs linked to constructor `self`. |
| definition | `timer` (L32:C10) | *Correct* | *Correct* | Yes | — | — | — | Correct field definition. |
| definition | `dt` (L19:C28) | *Correct* | *Correct* | Yes | — | — | — | Correct parameter definition. |
| definition | `shape` (L33:C39) | *Correct* | *Correct* | Yes | — | — | — | Correct field definition. |
| definition | `shapes` (L47:C12) | *Correct* | *Correct* | Yes | — | — | — | Correct local variable definition. |
| definition | `shape` (L52:C24) | *Correct* | *Incorrect* | No | — | — | — | uLangs linked to field `self.shape`. |
| definition | `50` (L13:C18) | *None* | *None* | Yes | — | — | — | Correctly no definition for literal. |
| references | `GameObject` (L3:C10) | 7 Refs | 7 Refs | — | 1.00 | 1.00 | 1.00 | Perfect class references. |
| references | `GameObject.new` (L3:C21) | 1 Ref | 1 Ref | — | 1.00 | 1.00 | 1.00 | Perfect method reference. |
| references | `area` (L7:C17) | 2 Refs | 4 Refs | — | 0.50 | 1.00 | 0.67 | 2 FPs (unrelated field). |
| references | `x` (L8:C22) | 2 Refs | 5 Refs | — | 0.40 | 1.00 | 0.57 | 3 FPs (2 unrelated `object.x`). |
| references | `timer` (L20:C13) | 4 Refs | 5 Refs | — | 0.80 | 1.00 | 0.89 | 1 FP (unrelated `love.timer`). |
| references | `shape` (L33:C39) | 6 Refs | 14 Refs | — | 0.43 | 1.00 | 0.60 | 8 FPs (local vars named `shape`). |
| references | `shapes` (L47:C12) | 3 Refs | 3 Refs | — | 1.00 | 1.00 | 1.00 | Perfect local var references. |
| references | `tag` (L40:C25) | 2 Refs | 3 Refs | — | 0.67 | 1.00 | 0.80 | 1 FP (unrelated `shape.tag`). |
| references | `HC.remove` (L33:C27) | 1 Ref | *None* | — | N/A | 0.00 | 0.00 | 1 FN (missed library call). |
| references | `object` (L52:C15) | 14 Refs | 15 Refs | — | 0.93 | 1.00 | 0.97 | 1 FP (initializer expression). |

Table 13.3: LSP Performance Evaluation of **Naive LSP Client** on Lua Code (*GameObject.lua*)

| Method | Target Symbol (Trigger Location) | Golden Result | Naive Result | Accurate | P | R | F1 | Notes |
|---|---|---|---|---|---|---|---|---|
| definition | `GameObject` (L3:C10) | *Correct* | *Correct* | Yes | — | — | — | Correct class definition. |
| definition | `area` (L7:C17) | *Correct* | *Correct* | Yes | — | — | — | Correct parameter definition. |
| definition | `GameObject` (L19:C10) | *Correct* | *Correct* | Yes | — | — | — | Correct method class context. |
| definition | `self` (L20:C8) | *Correct* | *Incorrect* | No | — | — | — | uLangs linked to constructor `self`. |
| definition | `timer` (L32:C10) | *Correct* | *Incorrect* | Yes | — | — | — | Correct field definition. |
| definition | `dt` (L19:C28) | *Correct* | *Correct* | Yes | — | — | — | Correct parameter definition. |
| definition | `shape` (L33:C39) | *Correct* | *Incorrect* | Yes | — | — | — | Correct field definition. |
| definition | `shapes` (L47:C12) | *Correct* | *Correct* | Yes | — | — | — | Correct local variable definition. |
| definition | `shape` (L52:C24) | *Correct* | *Incorrect* | No | — | — | — | uLangs linked to field `self.shape`. |
| definition | `50` (L13:C18) | *None* | *None* | Yes | — | — | — | Correctly no definition for literal. |
| references | `GameObject` (L3:C10) | 7 Refs | 7 Refs | — | 1.00 | 1.00 | 1.00 | Perfect class references. |
| references | `GameObject.new` (L3:C21) | 1 Ref | 1 Refs | — | 1.00 | 1.00 | 1.00 | Perfect method reference. |
| references | `area` (L7:C17) | 2 Refs | 4 Refs | — | 0.50 | 1.00 | 0.67 | 2 FPs (unrelated field). |
| references | `x` (L8:C22) | 2 Refs | 5 Refs | — | 0.40 | 1.00 | 0.57 | 3 FPs (2 unrelated `object.x`). |
| references | `timer` (L20:C13) | 4 Refs | 5 Refs | — | 0.80 | 1.00 | 0.89 | 1 FP (unrelated `love.timer`). |
| references | `shape` (L33:C39) | 6 Refs | 14 Refs | — | 0.43 | 1.00 | 0.60 | 8 FPs (local vars named `shape`). |
| references | `shapes` (L47:C12) | 3 Refs | 3 Refs | — | 1.00 | 1.00 | 1.00 | Perfect local var references. |
| references | `tag` (L40:C25) | 2 Refs | 3 Refs | — | 0.67 | 1.00 | 0.80 | 1 FP (unrelated `shape.tag`). |
| references | `HC.remove` (L33:C27) | 1 Ref | 1 Refs | — | 1.00 | 1.00 | 1.00 | Perfect for library call. |
| references | `object` (L52:C15) | 14 Refs | 15 Refs | — | 0.93 | 1.00 | 0.97 | 1 FP (initializer expression). |

**Notes for Table 13.2 and Table 13.3:** *Target Symbol* shows the symbol and the `line:column (L:C)` of request trigger (1-based). *Golden Result*, *uLangs Result* and *Naive Result* show the number of references found for `references` method, or the correctness for definitions. P=Precision, R=Recall, F1=F1-score. N/A indicates metric is undefined.

- **Macro-averaged scores**
  - **Average Precision:** 67.3%
  - **Average Recall:** 90.0%
  - **Average F1-Score:** 74.92%

## 13.4 Analysis and Interpretation

The evaluation of the uLangs output on an unseen Lua file (`GameObject.lua`) provides valuable insights into the current capabilities and limitations of the automated synthesis approach. Overall, for the `textDocument/definition` requests, uLangs achieved accuracy of **80%** (8 out of 10 correct). For the `textDocument/references` requests, the system demonstrated high recall (Micro-averaged: **97.6%**, Macro-averaged: **90.0%**) but more moderate precision (Micro-averaged: **71.9%**, Macro-averaged: **67.3%**), resulting in F1-scores of **82.8%** (Micro) and **74.9%** (Macro).

### 13.4.1 Go to Definition Performance

The **80% accuracy** for "Go to Definition" requests is a promising result for a system where semantic understanding is entirely synthesized. uLangs successfully identified the correct definitions for a variety of symbol types, including:

- Class declarations (e.g., `GameObject` at L3:C10).
- Function parameters (e.g., `area` at L7:C17, `dt` at L19:C28).
- Field declarations when the field itself was queried (e.g., `timer` at L32:C10, `shape` at L33:C39).
- Local variable declarations (e.g., `shapes` at L47:C12).
- Correctly determining that a literal value (`50` at L13:C18) has no definition.

The two failures in definition lookup highlight specific challenges for the synthesized semantic adapter:

1. `self` (in `GameObject:update` at L20:C8): The golden standard (lua-language-server) correctly identifies `self` in an instance method as referring to an instance of the `GameObject` class, thus pointing to the class definition. uLangs, however, incorrectly linked this usage of `self` to the `self` variable being modified within the constructor's options loop (`for k, v in pairs(opts) do self[k] = v end end`). This suggests a difficulty in distinguishing the implicit instance context of `self` in methods from its explicit use as a table in the constructor, potentially due to an oversimplified understanding of Lua's object-oriented patterns or a reliance on lexical proximity.
2. `shape` (parameter in anonymous function within `enemyProjectileCollisions` at L52:C24): The golden standard correctly identifies this `shape` as the local loop variable introduced by `for _, shape in ipairs(self:enter('Projectile')) do`. uLangs incorrectly linked it to the class field `self.shape`. This is a clear instance of scope resolution failure, where the synthesized adapter prioritized a broader class-level scope (the field `self.shape`) over the more immediate and correct local scope of the loop variable.

These errors indicate that while the adapter can handle many common definition types, but its understanding of more nuanced scoping rules, particularly concerning instance contexts (`self`) and the shadowing of names by local variables, requires further refinement.

### 13.4.2 Find References Performance

The evaluation of "Find References" provides a nuanced picture when comparing the uLangs-synthesized system against both the golden standard (`lua-language-server`) and the naive string-matching baseline.

The naive baseline, by its very design of finding all textual occurrences of a token identified by a regular expression (`[a-zA-Z_][a-zA-Z0-9_]*`), achieved perfect recall (**100%**) against the golden standard. Its precision (Micro: **72.4%**, Macro: **77.3%**) and resulting F1-scores (Micro: **83.9%**, Macro: **84.9%**) reflect its nature: it finds everything that matches lexically, including true references and numerous false positives where the same string appears in an unrelated context. While effective for Lua's simple identifier patterns, this regex-based token identification is inherently fragile. It would not generalize well to languages with different lexical rules for identifiers, making the naive baseline's strong performance somewhat language-dependent.

uLangs, in contrast, demonstrated very high recall (Micro: **97.6%**, Macro: **90.0%**), indicating it successfully identifies the vast majority of true references. This near-perfect recall, achieved through semantic analysis of an ANTLR-parsed AST, shows that its underlying pattern recognition for potential symbol occurrences is robust. Unlike the naive baseline, uLangs's symbol identification is based on a language-specific lexer and parser synthesized for Lua, providing a much more accurate and reliable foundation for what constitutes a "symbol" according to the language's actual syntax.

However, uLangs' precision was more moderate (Micro: **71.9%**, Macro: **67.3%**), leading to F1-scores of **82.8%** (Micro) and **74.9%** (Macro). While these F1-scores are comparable to the naive baseline, the key difference lies in the **attempt at semantic filtering.** The precision issues in uLangs stem not from an inability to find textual matches, but from the current limitations of the synthesized Prolog adapter in perfectly disambiguating scopes and contexts. This is a significantly more complex task than the naive baseline's simple string search. The false positives generated by uLangs are a result of imperfect semantic rules, whereas the naive baseline's false positives are an inherent consequence of having no semantic understanding at all.

The single false negative for uLangs—failing to identify a reference to the external library call `HC.remove`—further highlights this distinction. The naive baseline identified this purely due to the string `HC.remove` being present. uLangs missed it because its semantic model, synthesized from single-file examples, currently lacks the awareness of external libraries or symbols not defined within the analyzed scope. This is not a failure of its lexical identification but a specific, addressable gap in its semantic understanding, particularly concerning inter-file dependencies (as discussed in Section 9.5). With an enhanced semantic model capable of recognizing external symbols, uLangs could potentially identify such references correctly.

While the naive baseline sets a high bar for recall through exhaustive lexical matching, its utility is limited by its lack of semantic understanding and fragile tokenization. uLangs, despite current precision challenges, demonstrates a strong ability to identify nearly all true references based on a proper syntactic understanding of the language. Its path to improvement lies in refining the synthesized semantic adapter to better filter these candidates based on accurate scope and context, a level

of sophistication unattainable by the naive approach. The existing high recall suggests that the foundation for finding relevant symbols is already largely in place; the ongoing work is to enhance the semantic precision.

**Common Patterns in False Positives (FPs) for References:** The majority of precision issues stem from a few recurring patterns:

1. **Scope Resolution Errors:** This was the most dominant theme. The adapter frequently confused symbols with the same name but in different scopes or belonging to different entities.
    - For the parameter `area` (L7:C17), uLangs included the field `self.area` (LHS of assignment) and an unrelated `object.area` from a different part of the code.
    - For the parameter `x` (L8:C22), it included `self.x` (LHS) and two instances of `object.x` from the `enemyProjectileCollisions` method.
    - For the field `self.timer` (L20:C13), it included a reference to the global `love.timer`.
    - The query for the field `self.shape` (L33:C39) was particularly problematic, yielding 8 false positives, all of which were instances of local loop variables or parameters also named `shape` (e.g., `for shape in pairs(...)`, `if shape.tag == tag`). This highlights a critical failure to distinguish between class fields and local variables.
    - For the parameter `tag` (L40:C25), uLangs included the field `shape.tag`.
2. **LHS vs. RHS Confusion:** In several cases (e.g., parameters `area` and `x`), the adapter incorrectly identified the left-hand side (LHS) of an assignment (i.e., the field being assigned to, like `self.area` or `self.x`) as a reference to the parameter on the right-hand side (RHS). This suggests a potential lack of distinction in the adapter's logic between a symbol's definition/assignment site and its usage as a value.
3. **Over-reliance on Lexical Matching:** Many false positives appear to result from matching symbol names without a sufficiently deep semantic understanding of their distinct scopes, types, or the objects they belong to (e.g., `object.x` being confused with the parameter `x`).
4. **Initializer Expression as Reference:** For the local variable `object` (L52:C15), its initializer expression on the RHS (`shape.object`) was incorrectly included as a reference to the variable `object` itself.

**Analysis of False Negatives (FNs) for References** The evaluation revealed only a single instance of a false negative in the "Find References" task for uLangs:

- `HC.remove` (L33:C27): uLangs reported no references for this symbol. In contrast, the `lua-language-server` (golden standard) correctly identified the call site itself as a reference. The token `HC` likely refers to an external library or module whose definition is not present within the `GameObject.lua` file, and therefore not part of the semantic model synthesized by uLangs from single-file examples.

This specific failure highlights a current limitation in uLangs's semantic understanding: its inability to recognize or handle symbols that originate from outside the currently analyzed file or whose definitions are not captured within the synthesized LDS. The synthesized Prolog adapter is primarily geared towards finding references to symbols whose declarations *are* known within its model. It does not yet possess the mechanisms to infer the nature of, or find references to, symbols imported or defined externally. This points directly to the need for future work on inter-file dependency analysis and the *Project Adapter* concept (discussed in Section 9.5) to build a more comprehensive, project-wide semantic understanding. Notably, the naive string-matching baseline, by its very nature, identified this occurrence correctly—matching the golden standard—as it simply found the textual presence of `HC.remove` at the call site itself. This underscores the difference between the naive baseline's lexical pattern matching and uLangs's current, more constrained, model-based semantic approach for unresolvable external symbols.

### 13.4.3 Connection to the Synthesis Process and Semantic Adapter Capabilities

These results offer insights into the LLM-driven synthesis process for the semantic adapter:

- The **high recall** in reference finding might be a consequence of the LLM generating somewhat general Prolog rules that are effective at identifying potential symbol occurrences based on name and basic syntactic context. Prolog's pattern-matching capabilities are well-suited for this.
- The **precision issues** strongly indicate that the synthesized Prolog adapter currently lacks the sophisticated, language-specific rules needed for robust **scope disambiguation**, context understanding (LHS vs. RHS, parameter vs. field), and handling Lua's particular object-oriented mechanisms (like the contextual meaning of `self`).
- As theorized in Section 12.5, generating Prolog code to map arbitrary syntactic structures to a specific semantic schema is a novel and highly specialized task for LLMs. The training data for such a task is likely sparse compared to, for instance, generating Python code. This could explain why even the more capable `o3` model, used for adapter synthesis, struggled to infer these complex semantic rules flawlessly from the provided examples.
- The feedback mechanisms described in Section 9.4, including schema validation and the LLM-as-a-judge scorer (with its `MINIMUM_JUDGE_SCORE` threshold), are designed to improve semantic accuracy. While these help ensure structural correctness and basic plausibility, the current evaluation suggests they may not yet be sufficiently nuanced or stringent to guide the LLM to resolve the subtle scope and context errors that lead to false positives in reference finding.

### 13.4.4 Comparison to a Naive Baseline

To further contextualize the performance of the uLangs-synthesized Lua LDS, we compared it against a deliberately simplistic baseline implementation. This naive baseline relies purely on regular-expression-based string matching to provide the same LSP features: *Go to Definition* and *Find References*. Specifically, for definition requests,

it returns the first textual occurrence of the queried token in the file; for reference requests, it returns all textual occurrences of the token.

For the `textDocument/definition` requests, uLangs significantly outperformed the naive baseline, achieving an accuracy of **80%** compared to the baseline's **60%**. This difference highlights uLangs' semantic advantage: its synthesized Prolog adapter leverages the AST and semantic rules to accurately identify symbol definitions, rather than relying solely on textual occurrence.

For the `textDocument/references` requests, the naive baseline achieved perfect recall (**100%**) and relatively high precision (Micro: **72.4%**, Macro: **77.3%**), resulting in strong F1-scores (Micro: **83.9%**, Macro: **84.9%**). This high recall is expected, as the naive baseline simply identifies every textual occurrence of the token without semantic filtering. uLangs also demonstrated very high recall (Micro: **97.6%**, Macro: **90.0%**), closely matching the naive baseline. However, its precision was somewhat lower (Micro: **71.9%**, Macro: **67.3%**), leading to slightly lower F1-scores (Micro: **82.8%**, Macro: **74.9%**).

The naive baseline's effectiveness hinges on a carefully made regular expression for token identification. While suitable for Lua identifiers, this approach is fragile and language-specific. Languages with different lexical rules or more complex identifier patterns would likely significantly degrade the naive baseline's performance. In contrast, uLangs employs an ANTLR-generated lexer and parser, providing robust, language-specific tokenization and parsing. This ensures accurate identification of individual tokens according to the actual language syntax, making uLangs' approach more generalizable and reliable across diverse languages.

The single false negative for uLangs (`HC.remove`) illustrates a specific semantic gap rather than a fundamental limitation. The naive baseline found this reference due to simple string matching, while uLangs missed it because its current semantic model, synthesized from single-file examples, lacks awareness of external library symbols. This limitation is addressable by extending uLangs' semantic model to handle inter-file dependencies and external symbols, as discussed in Section 9.5.

While the naive baseline provides a strong recall-oriented benchmark through simple textual matching, uLangs demonstrates clear semantic advantages, particularly in definition accuracy and robust tokenization. The structured semantic representation provided by uLangs offers a foundation for further refinement and semantic precision improvements, highlighting its potential to significantly surpass naive approaches in practical language tooling scenarios.

### 13.4.5 Implications and Future Directions

The evaluation demonstrates that the uLangs approach can successfully synthesize an LDS capable of providing foundational LSP features for Lua. "Go to Definition" is largely functional for many common cases, while "Find References" offers comprehensive coverage (high recall) at the cost of some irrelevant results (moderate precision).

The primary areas for improvement in the synthesized semantic adapter lie in:

1. **Enhanced Scope Resolution:** This is the most critical area. The adapter must develop a more sophisticated understanding of lexical scopes, class/instance scopes, and how symbols are resolved within these contexts in Lua.

2. **Improved Precision for References:** Directly linked to scope resolution, this also involves better distinguishing symbol definitions/assignments from their usages, and more accurately interpreting the context of a symbol's appearance.

3. **Handling of External/Library Dependencies:** To be broadly useful, the system needs a strategy for recognizing and potentially linking symbols from external modules or libraries, aligning with the ideas proposed in Section 9.5 regarding inter-file dependencies and a "Project Adapter."

This case study helps to validate the core premise of this thesis: that language server components can be automatically synthesized from code examples and used as a useful code intelligence tool. The results clearly highlight the current strengths, particularly in broad symbol identification, and pinpoint the specific semantic complexities where the synthesis process, especially for the Prolog adapter, requires further refinement and more nuanced guidance. The path forward involves enhancing the semantic adapter's ability to interpret **language-specific scoping and context rules** more accurately.

# Chapter 14

# Limitations and Future Work

While uLangs achieves promising results in automating language server synthesis, several key directions remain for extending its capabilities, improving performance, and moving toward production-ready usage. This chapter explores these areas by grouping them into three strategic themes.

## 14.1 Extending Generality and Expressiveness

### 14.1.1 Project-level Semantic Analysis

As described in Section 9.5, current semantic analysis is scoped to single files. Supporting inter-file relationships, including import graphs and cross-file symbol resolution, is critical for real-world language tooling. A proposed solution is a dedicated *Project Adapter* that establishes project-wide semantics and dependency contexts.

### 14.1.2 Indentation-sensitive Language Support

Section 12.1.2 discusses the challenge of synthesizing grammars for indentation-based languages like Python. Future iterations could package reusable indentation-handling logic using embedded ANTLR actions to simplify grammar generation and improve parse-tree accuracy.

### 14.1.3 Semantic Feedback Extensions

The current LLM-as-a-judge mechanism, as detailed in Section 9.4, provides a valuable general assessment of semantic plausibility. However, as discussed in Section 13.4, the evaluation revealed that this mechanism, in its current form, may not offer sufficiently granular feedback to effectively address specific semantic inaccuracies. In particular, subtle scope resolution errors and context misunderstandings, such as those leading to false positives in the "Find References" evaluation, highlight the need for more targeted semantic feedback.

To address these limitations, future work could extend the semantic feedback mechanism by enabling the LLM-as-a-judge to evaluate multiple semantic dimensions independently. Rather than providing a single holistic score, the judge could separately assess aspects such as the correctness of symbol declarations, the accuracy of reference resolution, the completeness of symbol extraction, and the consistency of scope handling. Each of these dimensions could have its own configurable threshold, allowing the

synthesis process to pinpoint specific semantic weaknesses and guide the LLM toward more precise corrections.

Additionally, developing a generalized scoring schema would allow the semantic feedback mechanism to adapt more easily to new semantic queries beyond the current hardcoded holotype query. Such a schema would facilitate the introduction of additional LSP features or custom semantic analyses without an extensive redesign of the judging mechanism. For example, if type inference or inter-file dependency resolution were introduced, the judge could be equipped with criteria tailored specifically to evaluate these new semantic tasks.

Finally, the semantic feedback mechanism could be enhanced by training or prompting the LLM-as-a-judge to recognize common error patterns in the semantic adapter's output. By identifying recurring mistakes, such as confusing local variables with class fields or incorrectly resolving the context of special identifiers like `self`), the judge could provide more constructive and actionable feedback. This approach would transform the semantic feedback from a general quality check into a precise diagnostic tool, enabling more effective iterative refinement of the semantic adapter and significantly improving the accuracy and reliability of synthesized language server features.

### 14.1.4 Agentic Synthesis with Tool-Use

Future enhancements to uLangs could explore more sophisticated agentic workflows (See Section 3.3) to further empower the LLM during the synthesis of the LDS. Instead of passively responding to external verification, an agentic LLM could be granted direct access to a suite of tools within a controlled environment. This environment might allow the LLM to:

- Invoke the ANTLR toolchain to compile and test its generated grammars against code examples, directly observing parsing outcomes and errors.
- Execute its synthesized Prolog adapters with linearized facts and main queries, then analyze the resulting JSON for schema conformance and semantic plausibility.
- Potentially interact with the target language's actual compiler or interpreter (if available) to validate certain semantic assumptions or observe language behavior on test snippets.

By enabling the LLM to formulate hypotheses, test its own generated artifacts using these tools, and analyze the results, it could engage in a more autonomous cycle of self-correction and exploration. Such an agentic approach, leveraging the advancing capabilities of reasoning models, could lead to faster convergence on correct solutions, improved robustness in handling complex language features.

## 14.2 Fine-tuning

Currently, uLangs relies on general-purpose, pre-trained reasoning models (such as OpenAI's o3-mini accessed via APIs), combined with sophisticated prompting strategies and iterative feedback loops. While this approach has proven effective, it also presents clear opportunities for improvement. One particularly promising avenue for future work is the fine-tuning of specialized language models tailored explicitly to the tasks of grammar and semantic adapter synthesis.

Fine-tuning involves further training a pre-trained model on a carefully curated dataset specific to the target tasks. By fine-tuning models specifically for uLangs' synthesis tasks, we could significantly enhance the synthesis pipeline's accuracy and efficiency. A specialized model could internalize the nuances and common patterns inherent in ANTLR grammar generation and Prolog adapter creation, potentially leading to higher-quality initial candidates, fewer required repair iterations, and improved handling of complex language features or semantic rules.

Beyond accuracy, fine-tuning could also offer considerable efficiency gains. By leveraging smaller, open-weight models, such as variants of Llama, Mistral, or DeepSeek, we could enable local deployment or use more cost-effective inference APIs. Specialized models might also converge more quickly to correct solutions, reducing both latency and overall computational costs. Additionally, a fine-tuned model could simplify the prompting process, as much of the domain-specific knowledge currently embedded in detailed prompts would instead be encoded directly into the model's learned parameters.

To maximize the benefits of fine-tuning, we could consider creating distinct models specialized for each synthesis task. For example, one model could be fine-tuned specifically for grammar synthesis, trained to directly generate valid and generalized ANTLR grammar files from provided code examples. Another model could focus exclusively on semantic adapter synthesis, trained to produce correct and efficient Prolog adapter code given linearized parse tree facts and target JSON schemas.

However, fine-tuning specialized models introduces significant data requirements and practical challenges. The primary prerequisite is the creation of a high-quality, relatively large dataset tailored to each synthesis task. Potential data sources include existing open-source ANTLR grammars paired with representative code examples, manually curated examples with corresponding "golden" grammars and adapters, or even bootstrapped datasets generated by the current uLangs system itself, subsequently verified and corrected manually. Additionally, fine-tuning requires considerable computational resources and good evaluation metrics specifically designed to assess model performance on synthesis tasks.

Creating a dataset for Prolog adapter synthesis seems more challenging, as this type of semantic adapter code is novel and highly specific to uLangs. Such Prolog adapter code is likely underrepresented (or even absent), in the training data of current general-purpose LLMs. Consequently, bootstrapping examples using the current uLangs pipeline, followed by manual verification and correction, appears to be the most viable approach. Despite these challenges, this also means that fine-tuning could yield particularly significant improvements in adapter synthesis, as the model would gain specialized knowledge currently unavailable in general-purpose models.

Finally, a hybrid approach could also be explored. If smaller fine-tuned models prove effective at generating strong initial candidates but struggle with iterative repairs or complex edge cases, a hybrid pipeline could leverage fine-tuned models for initial generation and larger, general-purpose reasoning models for targeted repairs.

### 14.2.1 Reinforcement learning

Given that the artifacts we aim to generate (ANTLR grammars and Prolog adapters) are inherently verifiable, fine-tuning with reinforcement learning presents an intriguing avenue for fine-tuning specialized models.

Recent advancements in RL-based fine-tuning, such as the *Group Relative Policy Optimization* (GRPO) algorithm [40] used by DeepSeek in their R1 reasoning model [15], demonstrate how reward signals can effectively improve model training in domains like

mathematics and code. By leveraging the feedback signals already produced by uLangs' verification mechanisms, we could construct a reward model tailored specifically to our synthesis tasks. This reward model would directly incentivize the generation of grammars and adapters that pass verification checks, thereby systematically improving the model's ability to produce correct and robust artifacts.

Recent research [13] shows that smaller models, when fine-tuned using high-quality datasets and reinforcement learning techniques, can match or even outperform larger, more expensive models. Such an approach could significantly increase computational efficiency, enabling inexpensive yet performant artifact generation.

### 14.2.2 Automated Snippet Preparation

As discussed in Section 11.4.3, the synthesis pipeline depends on representative, well-scoped code snippets. Automating the extraction, splitting, and de-duplication of examples—potentially using LLM-driven in-context learning or heuristics—would reduce manual effort and improve synthesis coverage.

### 14.2.3 Context Compression Strategies

Section 11.3 proposes conservative context trimming to manage token budgets. Future work could explore LLM-based state summarization or semantic compression, enabling broader input coverage while maintaining synthesis reliability.

## 14.3 Enabling Production-Ready Tooling

### 14.3.1 Language Description Packaging

As noted in Section 7.2.1, the current Language Description Specification consists of multiple loose artifacts. Packaging these into structured archives or manifest-based bundles (e.g., ZIP + metadata) would simplify reuse, versioning, and integration into IDE tooling.

### 14.3.2 Long-lived process for LSP

An improvement for making uLangs production-ready is adding support for a true long-lived process model. Currently, each LSP request spawns a new subprocess to handle parsing and semantic analysis, which simplifies debugging but introduces significant overhead and limits responsiveness.

The main drawback of this approach is the repeated regeneration and recompilation of the ANTLR parser runtime from grammar files on every request, which is a costly code generation step. Moving to a persistent process would allow the parser to be generated and loaded once at startup, then reused for all following requests, greatly improving performance.

A long-lived process would also enable further optimizations, such as caching parsed files and their trees, which is especially beneficial when working with multiple files or large projects.

### 14.3.3 Partial code examples

A notable limitation of the current uLangs implementation arises during inference mode, particularly in interactive scenarios such as IDE usage, where users frequently work with incomplete or syntactically incorrect code. The synthesis pipeline was primarily designed and tested with complete, well-formed code examples, and thus its behavior with partial or unfinished code has not been extensively explored.

In practice, the ANTLR parsing component is likely to handle incomplete code reasonably well, thanks to its built-in error recovery mechanisms. However, the resulting parse tree will typically contain error nodes or incomplete branches at the points where parsing fails. The main challenge emerges in the semantic analysis phase: the Prolog adapter logic, synthesized from complete and correct parse trees, may behave unpredictably when encountering incomplete or erroneous input. Although the adapter's output must still conform to the required JSON schema, the semantic information it provides, especially for symbols near the incomplete or erroneous code, may be inaccurate or incomplete.

This limitation directly affects the reliability of IDE features such as go-to-definition, reference finding, or symbol listing, particularly when invoked near syntactic errors or incomplete code segments. Addressing this issue would likely require enhancements to the adapter synthesis process, potentially incorporating explicit error-handling strategies or specialized logic to gracefully manage incomplete syntactic structures. Addressing this challenge in potential future iterations would make uLangs-based language servers significantly more reliable and useful in real code editing environments.

# Chapter 15

# Conclusion

This thesis addressed the significant challenge of providing rich developer tooling for a growing number of programming languages, particularly domain-specific and niche languages, often underserved by manual LSP server development efforts. The creation of such tooling traditionally demands expertise and time, creating a barrier to enhanced developer productivity and language adoption. To this end, we introduced uLangs, a novel toolkit and methodology designed to automate the synthesis of core language server components—specifically ANTLR grammars for syntactic analysis and Prolog adapters for semantic interpretation—directly from code examples.

uLangs leverages LLM models within an iterative generate-verify-repair loop to produce a Language Description Specification. This specification, composed of the synthesized ANTLR and Prolog artifacts, then powers a universal LSP server, enabling essential code intelligence features such as 'Go to Definition' and 'Find References'. The core contributions of this work include the definition of this synthesizable LDS, the sophisticated synthesis logic using LLMs integrated with tools for verification, and the inference logic that uses the LDS for on-demand code intelligence.

The practical feasibility and effectiveness of uLangs were demonstrated through a detailed case study on the Lua programming language. The system successfully synthesized an ANTLR grammar capable of parsing 100% of the provided Lua snippets and a Prolog semantic adapter that correctly processed 93% of these snippets for semantic analysis. In functional LSP testing on an unseen Lua file, uLangs achieved an **80% accuracy** for 'Go to Definition' and an F1-score of 82.8% (micro-averaged) for 'Find References', showcasing its ability to generate genuinely useful language tooling components automatically from examples. These results showcase its ability to automatically generate genuinely useful language tooling components from examples, with the system learning to provide meaningful code intelligence without prior explicit knowledge of the given language. Interestingly, the synthesis process for the Lua case study, including both grammar and semantic adapter generation, incurred a modest operational cost (as detailed in Table 13.1, approximately $7.24 USD using OpenAI's o3-mini and o3 models), demonstrating the economic viability of this approach, particularly as LLM efficiency continues to improve. Throughout developing and testing uLangs (including all experiments, debugging, and iterative synthesis across multiple languages and scenarios) the total cost amounted to approximately $41 USD.

The journey of developing uLangs provided evidence that advancements in LLM capabilities directly translate to improved synthesis quality. Several initial roadblocks encountered during the project, particularly in generating robust semantic adapters, were often mitigated or resolved with the advent of more sophisticated reasoning models. This

experience strongly suggests that future, more powerful and nuanced LLMs will further enhance the accuracy of the synthesized artifacts, reduce the need for extensive iterative repair, and potentially enable uLangs to achieve near-perfect synthesis for a broader range of complex language constructs with even less manual guidance.

Furthermore, the design choice of utilizing ANTLR for grammars and Prolog for semantic adapters offers a significant advantage: the generated LDS artifacts are human-readable and -editable. Should the automated synthesis not achieve perfection, or if specific, nuanced behaviors are desired beyond what can be inferred from examples, developers familiar with these widely used formalisms can manually inspect, refine, or extend the LLM-generated code. This 'human-in-the-loop' capability ensures that uLangs can be a powerful bootstrapping tool, significantly reducing initial development effort, even if fully automated perfection remains elusive for the most intricate scenarios. The LLMs generally produce a solid foundation amenable to such manual adjustments.

In conclusion, this thesis has demonstrated that automated language server synthesis from code examples, as implemented by uLangs, is a viable and promising approach. Combining the generative power of LLMs with the structured verification of formal language tools, uLangs offers a novel pathway to significantly lower the barrier to creating sophisticated code intelligence. As outlined in Chapter 14, future work can further extend uLangs' capabilities toward production-ready tooling, addressing various languages and project complexities. Ultimately, the results presented in this thesis indicate that uLangs lives up to its name as a *Universal Language Server Toolkit*, showcasing its ability to effectively automate language tooling development, making sophisticated IDE features more readily achievable with minimal effort.

# Bibliography


[1] Hendrik van Antwerpen, Rob Rix, and Douglas A. Creager. *tree-sitter/tree-sitter-graph: v0.11.2*. Mar. 2024. DOI: 10.5281/zenodo.10798149. URL: https://zenodo.org/records/10798149 (visited on 05/16/2024).

[2] Yuntao Bai et al. *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. arXiv:2204.05862 [cs]. Apr. 2022. DOI: 10.48550/arXiv.2204.05862. URL: http://arxiv.org/abs/2204.05862 (visited on 04/29/2024).

[3] Marthe Ballon, Andres Algaba, and Vincent Ginis. *The Relationship Between Reasoning and Performance in Large Language Models – o3 (mini) Thinks Harder, Not Longer*. arXiv:2502.15631 [cs]. Feb. 2025. DOI: 10.48550/arXiv.2502.15631. URL: http://arxiv.org/abs/2502.15631 (visited on 03/22/2025).

[4] Bart Kiers. *python3-parser; an ANTLR4 grammar for Python 3*. July 2024. URL: https://github.com/antlr/grammars-v4/tree/c3553dea88dcef2f6183648e2e42bde340960237/python/python3 (visited on 03/22/2025).

[5] Osbert Bastani et al. "Synthesizing program input grammars." In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI 2017. New York, NY, USA: Association for Computing Machinery, June 2017, pp. 95–110. ISBN: 978-1-4503-4988-8. DOI: 10.1145/3062341.3062349. URL: https://dl.acm.org/doi/10.1145/3062341.3062349 (visited on 05/10/2025).

[6] Dominik Bork and Philip Langer. "Language Server Protocol: An Introduction to the Protocol, its Use, and Adoption for Web Modeling Tools." en. In: *Enterprise Modelling and Information Systems Architectures (EMISAJ)* 18 (Sept. 2023), 9:1–16. ISSN: 1866-3621. DOI: 10.18417/emisa.18.9. URL: https://emisa-journal.org/emisa/article/view/320 (visited on 04/28/2024).

[7] David Brandfonbrener et al. *VerMCTS: Synthesizing Multi-Step Programs using a Verifier, a Large Language Model, and Tree Search*. arXiv:2402.08147 [cs]. May 2024. DOI: 10.48550/arXiv.2402.08147. URL: http://arxiv.org/abs/2402.08147 (visited on 03/23/2025).

[8] Tom B. Brown et al. *Language Models are Few-Shot Learners*. arXiv:2005.14165 [cs]. July 2020. DOI: 10.48550/arXiv.2005.14165. URL: http://arxiv.org/abs/2005.14165 (visited on 04/29/2024).

[9] Cameron B. Browne et al. "A Survey of Monte Carlo Tree Search Methods." In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1 (Mar. 2012). Conference Name: IEEE Transactions on Computational Intelligence and AI in Games, pp. 1–43. ISSN: 1943-0698. DOI: 10.1109/TCIAIG.2012.2186810. URL: https://ieeexplore.ieee.org/document/6145622 (visited on 03/23/2025).

[10] Sébastien Bubeck et al. *Sparks of Artificial General Intelligence: Early experiments with GPT-4*. arXiv:2303.12712 [cs]. Apr. 2023. DOI: 10.48550/arXiv.2303.12712. URL: http://arxiv.org/abs/2303.12712 (visited on 04/29/2024).

[11] Yupeng Chang et al. *A Survey on Evaluation of Large Language Models*. arXiv:2307.03109 [cs]. Dec. 2023. DOI: 10.48550/arXiv.2307.03109. URL: http://arxiv.org/abs/2307.03109 (visited on 04/29/2024).

[12] Yuheng Cheng et al. *Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects*. arXiv:2401.03428 [cs]. Jan. 2024. DOI: 10.48550/arXiv.2401.03428. URL: http://arxiv.org/abs/2401.03428 (visited on 04/29/2024).

[13] Quy-Anh Dang and Chris Ngo. *Reinforcement Learning for Reasoning in Small LLMs: What Works and What Doesn't*. arXiv:2503.16219 [cs]. Mar. 2025. DOI: 10.48550/arXiv.2503.16219. URL: http://arxiv.org/abs/2503.16219 (visited on 04/21/2025).

[14] DeepSeek. *Models & Pricing | DeepSeek API Docs*. en. URL: https://api-docs.deepseek.com/quick_start/pricing (visited on 03/22/2025).

[15] DeepSeek-AI et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948 [cs]. Jan. 2025. DOI: 10.48550/arXiv.2501.12948. URL: http://arxiv.org/abs/2501.12948 (visited on 03/22/2025).

[16] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997 [cs]. Mar. 2024. DOI: 10.48550/arXiv.2312.10997. URL: http://arxiv.org/abs/2312.10997 (visited on 04/29/2024).

[17] Github Team. *GitHub Copilot · Your AI pair programmer*. en. 2025. URL: https://github.com/features/copilot (visited on 05/09/2025).

[18] Maliheh Izadi et al. *Language Models for Code Completion: A Practical Evaluation*. arXiv:2402.16197 [cs]. Feb. 2024. DOI: 10.48550/arXiv.2402.16197. URL: http://arxiv.org/abs/2402.16197 (visited on 04/26/2024).

[19] JetBrains. *How Does MPS Work? - Concepts | MPS by JetBrains*. en. URL: https://www.jetbrains.com/mps/concepts/ (visited on 03/26/2025).

[20] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. "Soufflé: On Synthesis of Program Analyzers." In: *Computer Aided Verification*. Ed. by Swarat Chaudhuri and Azadeh Farzan. Cham: Springer International Publishing, 2016, pp. 422–430. ISBN: 978-3-319-41540-6.

[21] Donald E. Knuth. "Semantics of context-free languages." In: *Mathematical systems theory* 2.2 (June 1968), pp. 127–145. ISSN: 1433-0490. DOI: 10.1007/BF01692511. URL: https://doi.org/10.1007/BF01692511.

[22] Neil Kulkarni, Caroline Lemieux, and Koushik Sen. *Learning Highly Recursive Input Grammars*. arXiv:2108.13340 [cs]. Aug. 2021. DOI: 10.48550/arXiv.2108.13340. URL: http://arxiv.org/abs/2108.13340 (visited on 05/10/2025).

[23] Feifei Li et al. *Incremental Context-free Grammar Inference in Black Box Settings*. arXiv:2408.16706 [cs]. Sept. 2024. DOI: 10.48550/arXiv.2408.16706. URL: http://arxiv.org/abs/2408.16706 (visited on 05/10/2025).

[24] Tianle Li et al. *Long-context LLMs Struggle with Long In-context Learning*. arXiv:2404.02060 [cs]. June 2024. DOI: 10.48550/arXiv.2404.02060. URL: http://arxiv.org/abs/2404.02060 (visited on 04/29/2025).

[25] Kari Lukka. "The Constructive Research Approach." In: *Case Study Research in Logistics*. Journal Abbreviation: Case Study Research in Logistics. Jan. 2003, pp. 83–101.

[26] Michelle Pokrass. *Introducing Structured Outputs in the API*. en-US. Aug. 2024. URL: https://openai.com/index/introducing-structured-outputs-in-the-api/ (visited on 03/22/2025).

[27] Ali Modarressi et al. *NoLiMa: Long-Context Evaluation Beyond Literal Matching*. arXiv:2502.05167 [cs]. Mar. 2025. DOI: 10.48550/arXiv.2502.05167. URL: http://arxiv.org/abs/2502.05167 (visited on 04/29/2025).

[28] C. G. Nevill-Manning and I. H. Witten. *Identifying Hierarchical Structure in Sequences: A linear-time algorithm*. arXiv:cs/9709102. Sept. 1997. DOI: 10.48550/arXiv.cs/9709102. URL: http://arxiv.org/abs/cs/9709102 (visited on 05/10/2025).

[29] OpenAI. *OpenAI o3-mini*. en-US. Mar. 2025. URL: https://openai.com/index/openai-o3-mini/ (visited on 03/22/2025).

[30] OpenAI. *Pricing*. Mar. 2023. URL: https://web.archive.org/web/20230320175036/https://openai.com/pricing (visited on 03/23/2025).

[31] OpenAI. *Pricing*. en-US. 2025. URL: https://openai.com/api/pricing/ (visited on 03/22/2025).

[32] OpenAI. *Prompt Caching in the API*. en-US. Oct. 2024. URL: https://openai.com/index/api-prompt-caching/ (visited on 03/23/2025).

[33] OpenAI et al. *OpenAI o1 System Card*. arXiv:2412.16720 [cs]. Dec. 2024. DOI: 10.48550/arXiv.2412.16720. URL: http://arxiv.org/abs/2412.16720 (visited on 03/17/2025).

[34] Terence Parr and Kathleen S Fisher. "LL(*): The Foundation of the ANTLR Parser Generator." en. In: (2011).

[35] Terence John Parr. *The Definitive ANTLR 4 Reference*. en. Google-Books-ID: YwuoAQAACAAJ. Pragmatic Bookshelf, 2012. ISBN: 978-1-941222-62-1.

[36] Mohaimenul Raiaan et al. *A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges*. Sept. 2023. DOI: 10.36227/techrxiv.24171183.

[37] Jonas Kjær Rask et al. "The Specification Language Server Protocol: A Proposal for Standardised LSP Extensions." In: *Electronic Proceedings in Theoretical Computer Science* 338 (Aug. 2021). arXiv:2108.02961 [cs], pp. 3–18. ISSN: 2075-2180. DOI: 10.4204/EPTCS.338.3. URL: http://arxiv.org/abs/2108.02961 (visited on 04/29/2024).

[38] Robert Shin and Arun Patro. *Monte Carlo Tree Search for Code Generation using LLMs*. Feb. 2024. URL: https://arunpatro.github.io/blog/mcts/ (visited on 03/23/2025).

[39] Stuart Jonathan Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. en. Google-Books-ID: CUVeMwAACAAJ. Prentice Hall, 1995. ISBN: 978-0-13-103805-9.

[40] Zhihong Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300 [cs]. Apr. 2024. DOI: 10.48550/arXiv.2402.03300. URL: http://arxiv.org/abs/2402.03300 (visited on 04/21/2025).

[41] Jiho Shin et al. *Prompt Engineering or Fine Tuning: An Empirical Assessment of Large Language Models in Automated Software Engineering Tasks*. arXiv:2310.10508 [cs]. Oct. 2023. DOI: 10.48550/arXiv.2310.10508. URL: http://arxiv.org/abs/2310.10508 (visited on 04/29/2024).

[42] Spoofax Team. *Developing Spoofax*. en. Jan. 2025. URL: https://www.spoofax.dev/howtos/development/develop-spoofax/ (visited on 03/26/2025).

[43] Spoofax Team. *ESV*. en. Jan. 2025. URL: https://www.spoofax.dev/references/esv/ (visited on 03/26/2025).

[44] Spoofax Team. *FlowSpec*. en. Jan. 2025. URL: https://www.spoofax.dev/references/flowspec/ (visited on 03/26/2025).

[45] Spoofax Team. *SDF3*. en. Jan. 2025. URL: https://www.spoofax.dev/references/sdf3/ (visited on 03/26/2025).

[46] Spoofax Team. *Statix*. en. Jan. 2025. URL: https://www.spoofax.dev/references/statix/ (visited on 03/26/2025).

[47] Spoofax Team. *Stratego*. en. Jan. 2025. URL: https://www.spoofax.dev/references/stratego/ (visited on 03/26/2025).

[48] Andrew Stevenson and James R. Cordy. "A survey of grammatical inference in software engineering." In: *Science of Computer Programming* 96 (2014), pp. 444–459. ISSN: 0167-6423. DOI: https://doi.org/10.1016/j.scico.2014.05.008. URL: https://www.sciencedirect.com/science/article/pii/S0167642314002469.

[49] SWI-Prolog Team. *Should I be using SWI-Prolog?* Dec. 2020. URL: https://www.swi-prolog.org/pldoc/man?section=swiorother (visited on 05/07/2025).

[50] Yashar Talebirad and Amirhossein Nadiri. *Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents*. arXiv:2306.03314 [cs]. June 2023. DOI: 10.48550/arXiv.2306.03314. URL: http://arxiv.org/abs/2306.03314 (visited on 04/29/2024).

[51] Terence Parr and ANTLR. *ANTLR*. URL: https://www.antlr.org/ (visited on 05/14/2025).

[52] Mohammad Jalili Torkamani. *Kajal: Extracting Grammar of a Source Code Using Large Language Models*. arXiv:2412.08842 [cs] version: 1. Dec. 2024. DOI: 10.48550/arXiv.2412.08842. URL: http://arxiv.org/abs/2412.08842 (visited on 05/10/2025).

[53] Bailin Wang et al. *Grammar Prompting for Domain-Specific Language Generation with Large Language Models*. arXiv:2305.19234 [cs]. Nov. 2023. DOI: 10.48550/arXiv.2305.19234. URL: http://arxiv.org/abs/2305.19234 (visited on 05/10/2025).

[54] Chengpeng Wang et al. *LLMSA: A Compositional Neuro-Symbolic Approach to Compilation-free and Customizable Static Analysis*. arXiv:2412.14399 [cs] version: 1. Dec. 2024. DOI: 10.48550/arXiv.2412.14399. URL: http://arxiv.org/abs/2412.14399 (visited on 05/10/2025).

[55] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv:2201.11903 [cs]. Jan. 2023. DOI: 10.48550/arXiv.2201.11903. URL: http://arxiv.org/abs/2201.11903 (visited on 05/16/2024).

[56] Xtext Team. *Xtext - Language Engineering Made Easy!* URL: https://eclipse.dev/Xtext/ (visited on 05/08/2025).

[57] Xtext Team. *Xtext - The Grammar Language*. URL: https://eclipse.dev/Xtext/documentation/301_grammarlanguage.html (visited on 03/26/2025).

[58] Chenyuan Yang et al. *KNighter: Transforming Static Analysis with LLM-Synthesized Checkers*. arXiv:2503.09002 [cs]. Apr. 2025. DOI: 10.48550/arXiv.2503.09002. URL: http://arxiv.org/abs/2503.09002 (visited on 05/10/2025).

[59] Shengyu Zhang et al. *Instruction Tuning for Large Language Models: A Survey*. arXiv:2308.10792 [cs]. Mar. 2024. DOI: 10.48550/arXiv.2308.10792. URL: http://arxiv.org/abs/2308.10792 (visited on 04/29/2024).

[60] Zod Team. *TypeScript-first schema validation with static type inference*. en. URL: https://github.com/colinhacks/zod (visited on 05/10/2025).

# Appendix A

# Prompts

```
ANTLR (Another Tool for Language Recognition) uses **context-free grammars (
    CFGs)** to define languages and generate parsers. ANTLR4 specifically
    employs **LL(*) parsing**a variation of **LL(k)** parsers that can handle
    arbitrary lookahead. Heres a breakdown of the logic required to follow
    ANTLR4 specs:

### 1. **Understanding LL(*) Parsers:**
- **Top-down** parsing: ANTLR generates parsers that parse input from the top-
    most rule (start rule) down to the leaves.
- **Predictive parsing**: It uses lookahead to choose between alternatives,
    avoiding backtracking when possible.
- **Non-determinism handling**: ANTLR resolves ambiguities by analyzing the
    grammar and employing **adaptive lookahead**.

### 2. **Grammar Structure (EBNF-like):**
- **Lexer rules** (Uppercase names): Define tokens by matching regular
    expressions.
- **Parser rules** (Lowercase names): Define how tokens combine to form higher-
    level constructs.
- **Actions**: Embedded code (usually in Java or other target languages)
    executes within the grammar.

antlr
grammar Expr;

expr: expr ('*'|'/') expr  # MulDiv
    | expr ('+'|'-') expr  # AddSub
    | INT                  # Int
    | '(' expr ')'         # Parens
    ;

INT: [0-9]+;
WS: [ \t\r\n]+ -> skip;


### 3. **Parse Tree Logic:**
- ANTLR builds a **parse tree** where each node corresponds to a grammar rule.
- The parser recursively descends through the tree, applying rules and matching
     tokens.
```

```
### 4. **Backtracking and Prediction:**
- ANTLR4 uses **prediction modes** (SLL, LL) to manage parsing paths.
- **Ambiguity resolution**: It prefers the longest match or the first
    alternative.

### 5. **Code Generation:**
- ANTLR generates parsers in languages like Java, Python, C#, and Go.
- The generated code implements listener/visitor patterns to traverse parse
    trees.

### Key Concepts to Master:
- **Tokenization** (lexer rules)
- **Recursive descent parsing** (parser rules)
- **Left-recursion elimination** (ANTLR4 supports direct left-recursion)
- **Error handling and recovery** (custom error strategies)
- **Semantic predicates** (condition parsing on semantic checks)
Ensuring all of this is correct when building a grammar is a challenge and
    requires extreme attention to detail.

Task:
From code snippet files, create the full ANTLR (antlr4) grammar to fully parse
    this programming language. Make sure all logic adds up and originates from
    the start rule. Create both a MyLexer.g4 and MyParser.g4 file. Start rule
    must be called "program". Make it somewhat generic as these files are just
    examples. Check that the grammar works for each file given at least. Don't
    add comment unless needed for an expert to understand the logic.
Always output the WHOLE and full grammar for both lexer and parser, within
    blocks like this  ‘‘‘antlr
```

Listing A.1: LLM prompt for ANTLR grammar generation.

```
Your goal is to generate a robust, correct, and optimal Prolog adapter that
    transforms a Concrete Syntax Tree (CST) into an Abstract Syntax Tree (AST),
     accurately extracting symbol information (declarations and references)
    suitable for Language Server Protocol (LSP) symbol listing.

## Adapter Goals
Your adapter MUST accomplish the following precisely and reliably:

- **Declarations:**
  Identify exactly which CST nodes represent symbol **declarations**. A
    declaration introduces a new named entity in the source code (variables,
    parameters, functions, classes, etc.).

- **References:**
  Identify exactly which CST nodes represent **references** to already-declared
     symbols. A reference mentions an existing symbol without redeclaring it.

- **Uniqueness:**
  Each declaration should appear **once** and should link accurately to all its
     corresponding references. No declaration duplicates should appear.

- **Ordering and Scoping:**
  Respect source-file ordering. A symbol is declared exactly once at the
```

```
    earliest occurrence, and all subsequent occurrences are references. If the
    language has special scoping rules (locals, globals, parameters, etc.),
    ensure your adapter logic correctly reflects those rules.

## Adapter Requirements
- You MUST only write within the provided WRITEABLE AREA.
- Output ONLY Prolog code. This code will be placed into an adapter file.
- Ensure that your code makes the provided Main query execute successfully and
    correctly.
- Do NOT emit incomplete or placeholder code, every emitted rule or predicate
    must have well-defined behavior and work as-is.

## Writing Instructions
- **Define helper predicates** first to simplify your main logic (e.g.,
    predicates like `earlier_in_file/2`, `is_declaration/3`, `is_reference/2`).
- **Implement clear logic rules**: a node is either a declaration or a
    reference, never both.
- **Explicitly handle symbol kinds** such as variables (13), functions (12),
    parameters (variable or parameter kind), and others as relevant to the CST
    provided.
- Write your adapter incrementally and test your reasoning by carefully
    considering counterexamples (e.g., shadowed names, reassigned variables,
    nested scopes).

You MUST only write within the WRITEABLE AREA. Output only the adapter /
    WRITEABLE AREA.
```

Listing A.2: LLM prompt for Prolog adapter generation.

```
function adapterScoringMessage(snippet: string, adapterOutput: string) {
    return `Score 0 to 100 how well the generated definitions and
    references is by looking at the code snippet it is based on. 100 =
    means that the definitions perfectly describe the code snippet. 0 =
    means that the output dont make ANY sense. 50 = means that the output
    is mostly correct, but there are some errors but should be passing.
You will also be given the AST as context.
Note: Ignore following: builtin functions, undefined symbols, etc.
Note: Ignore readability of the output, e.g., if the output is too
    verbose or too terse. The output has already passed the schema
    validation.
HEAVILY PENALIZE symbols, defintions and references that are missing,
    incorrect, or not relevant. BUT dont penalize impossible symbols to
    resolve by context.
* Do symbols/definitions have references?

Supply also a list of reasons/errors (empty list if there are none if
    **perfect**, it is used as feedback).
<CodeSnippet>
${snippet}
</CodeSnippet>
<GeneratedOutput>
${adapterOutput}
</GeneratedOutput>`;
}
```

Listing A.3: Dynamic LLM prompt function for scoring of Prolog adapters.

# Appendix B

# JSON Schema

```
import { z } from 'zod';

const LocationSchema = z.object({
  col: z.number(),
  file: z.string(),
  len: z.number(),
  line: z.number(),
});

const ReferenceSchema = LocationSchema;

const SymbolSchema = z.object({
  kind: z.number(),
  location: LocationSchema,
  name: z.string(),
  references: z.array(ReferenceSchema),
});

export const DefinitionQueryResultSchema = z.object({
  symbols: z.array(SymbolSchema),
});

export type Definition = z.infer<typeof SymbolSchema>;
export type DefinitionQueryResult = z.infer<typeof
    DefinitionQueryResultSchema>;
```

Listing B.1: Zod JSON Schema definition for the *definitions* main query

# Appendix C

# Case Study

```
GameObject = Object:extend()

function GameObject:new(area, x, y, opts)
    local opts = opts or {}
    if opts then for k, v in pairs(opts) do self[k] = v end end

    self.area = area
    self.x, self.y = x, y
    self.id = UUID()
    self.creation_time = love.timer.getTime()
    self.timer = Timer()
    self.dead = false
    self.depth = 50

    self.previous_collisions = {}
    self.current_collisions = {}
end

function GameObject:update(dt)
    if self.timer then self.timer:update(dt) end
    if self.shape then
        self.previous_collisions = table.copy(self.current_collisions)
        self.current_collisions = {}
    end
end

function GameObject:draw()

end

function GameObject:destroy()
    self.timer:destroy()
    if self.shape then HC.remove(self.shape) end
    self.shape = nil
end

function GameObject:enter(tag)
    local shapes = {}
    for shape in pairs(HC.neighbors(self.shape)) do
        if shape.tag == tag then
            if self.shape:collidesWith(shape) then
                self.current_collisions[shape.id] = true
                if not self.previous_collisions[shape.id] then
   table.insert(shapes, shape) end
            end
```

```
45          end
46      end
47      return shapes
48  end
49
50  function GameObject:enemyProjectileCollisions()
51      for _, shape in ipairs(self:enter('Projectile')) do
52          local object = shape.object
53          if object then
54              self:hit(object.damage, object.x, object.y, object.r)
55              if self.hp <= 0 then current_room.player:onKill(self) end
56              if object.pierce > 0 then
57                  object.pierce = object.pierce - 1
58                  if object.attack == 'Explode' then
    object.area:addGameObject('Explosion', object.x, object.y, {color =
    self.color}) end
59                  else object:die() end
60          end
61      end
62  end
```

Listing C.1: *GameObject.lua* Source Code in Lua from the *BYTEPATH* repository

# Appendix D

# Declaration of the use of generative artificial intelligence

In this scientific work, generative artificial intelligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

- I have extensively used generative AI (specifically GPT models provided by OpenAI) for brainstorming and exploring ideas during the initial phases of the project. This included generating and critiquing potential approaches, methodologies, and design decisions related to the uLangs toolkit.
- I employed AI assistance for iterative language editing and refinement throughout the thesis-writing process. Draft sections were repeatedly reviewed and improved using generative AI, involving multiple iterations and manual integration of suggestions to enhance clarity, readability, and coherence. All AI-generated suggestions were carefully scrutinized, fact-checked, and manually adjusted by me.
- I used generative AI to critique and refine the presentation of ideas, arguments, and technical explanations within the thesis. This included identifying unclear phrasing, logical inconsistencies, or areas requiring additional clarification.
- Generative AI was used during the software development phase of the uLangs toolkit. Specifically, it assisted me in debugging code, identifying potential implementation errors, and suggesting improvements or alternative solutions to technical challenges encountered during development.

All AI-generated content included in this thesis has been thoroughly scrutinized, reviewed, and manually validated by me. I fully stand behind all the work presented in this thesis, and I take complete responsibility for its accuracy, quality, and integrity.